Computer Vision and Machine Learning for Scientific Visual Data Analytics

Lecture 4: Deep Neural Networks

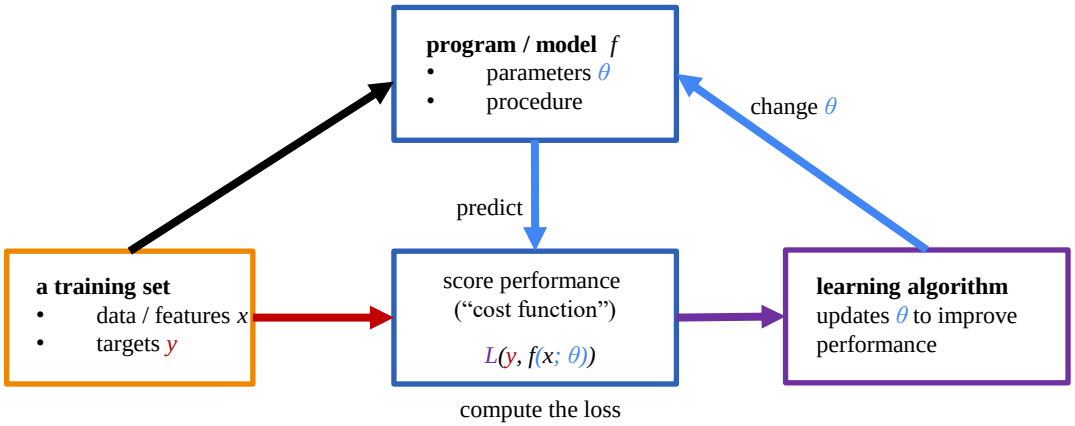
Shu Kong
shu.kong@oist.jp

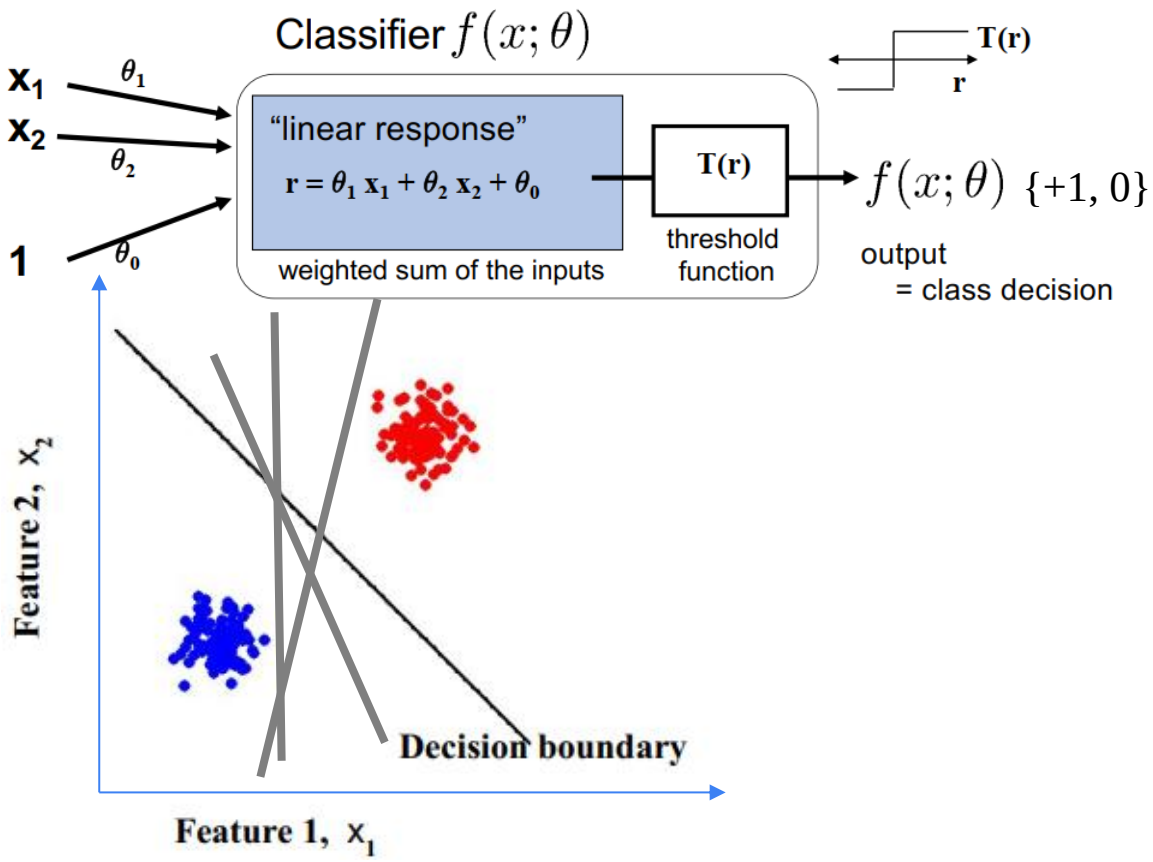
Roadmap

Teaching materials are available: https://github.com/aimerykong/OIST-mini-course-CVML

Lecture	Topics	Hands-on experience
1. Intro	CV/ML, deep learning, artificial intelligence	Configure programming environment using Google Drive and Colab; implement basic scripts in Python
2. Basics of CV/ML	training, validation, testing, features, metrics, nearest neighbor	Implement a machine learning pipeline for image classification
3. Classification	Logistic Regression, gradient descent	Train a linear classifier for pollen species recognition
4. Deep Learning	Multi-Layer Perceptron, feature learning, backpropagation, neural networks	Train a convolutional neural network (CNN) for pollen species recognition
5. Segmentation and Detection	semantic and instance segmentation, U-Net,	Train a neural network for pollen grain detection/segmentation
6. Foundation Models	CLIP, Segment Anything Model, DINOv2, GroundingDINO, finetuning	Finetune a pretrained model for image classification and segmentation
7. Advanced Topics	open-set recognition, novel species recognition, phylogentic reconstruction via clustering	Implement algorithms to recognize unknown/anomalous/novel observations
8. Brainstorm & Talks	Volunteer to present	n/a

Recap: Linear Classifier

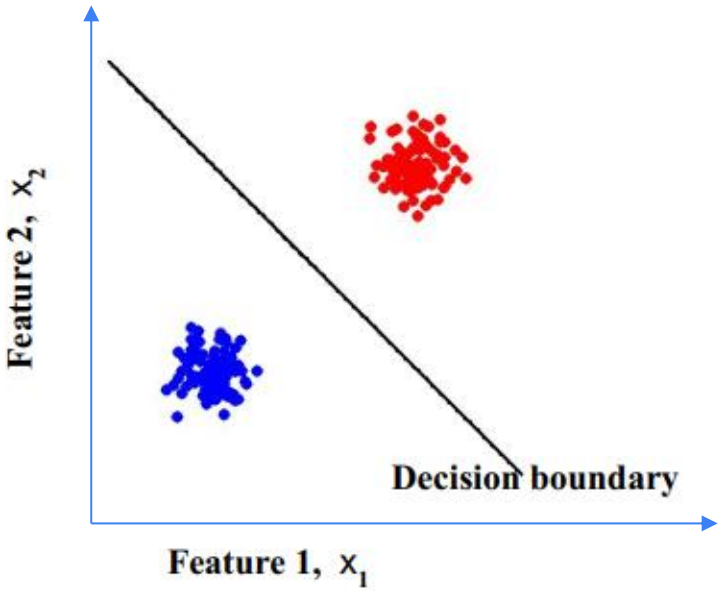




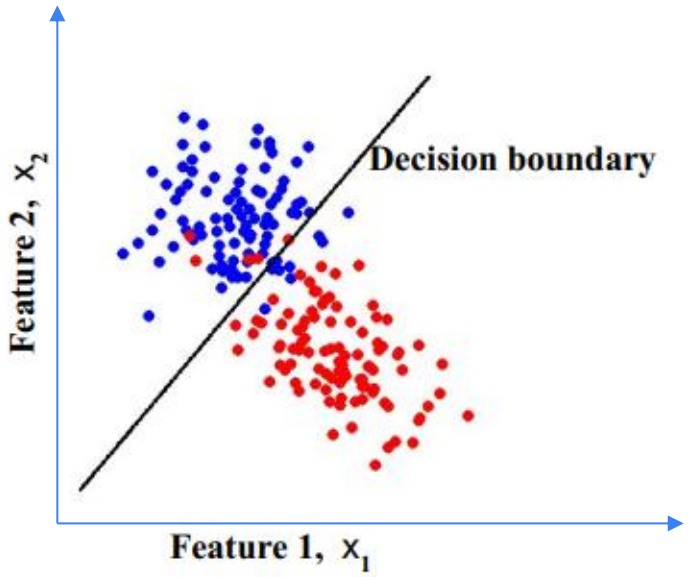
Separability

- A data set is separable by a learner if
  - There is some instance of that learner that correctly predicts all the data points
- Linearly separable data
  - Can separate the two classes using a straight line in feature space
  - in 2 dimensions the decision boundary is a straight line

Linearly separable data

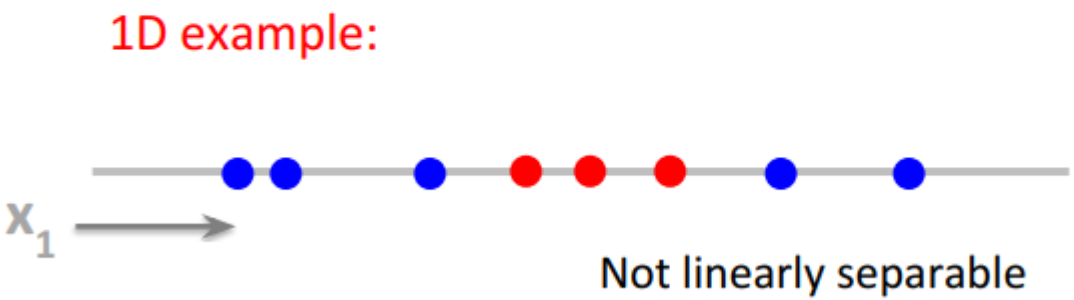


Linearly non-separable data



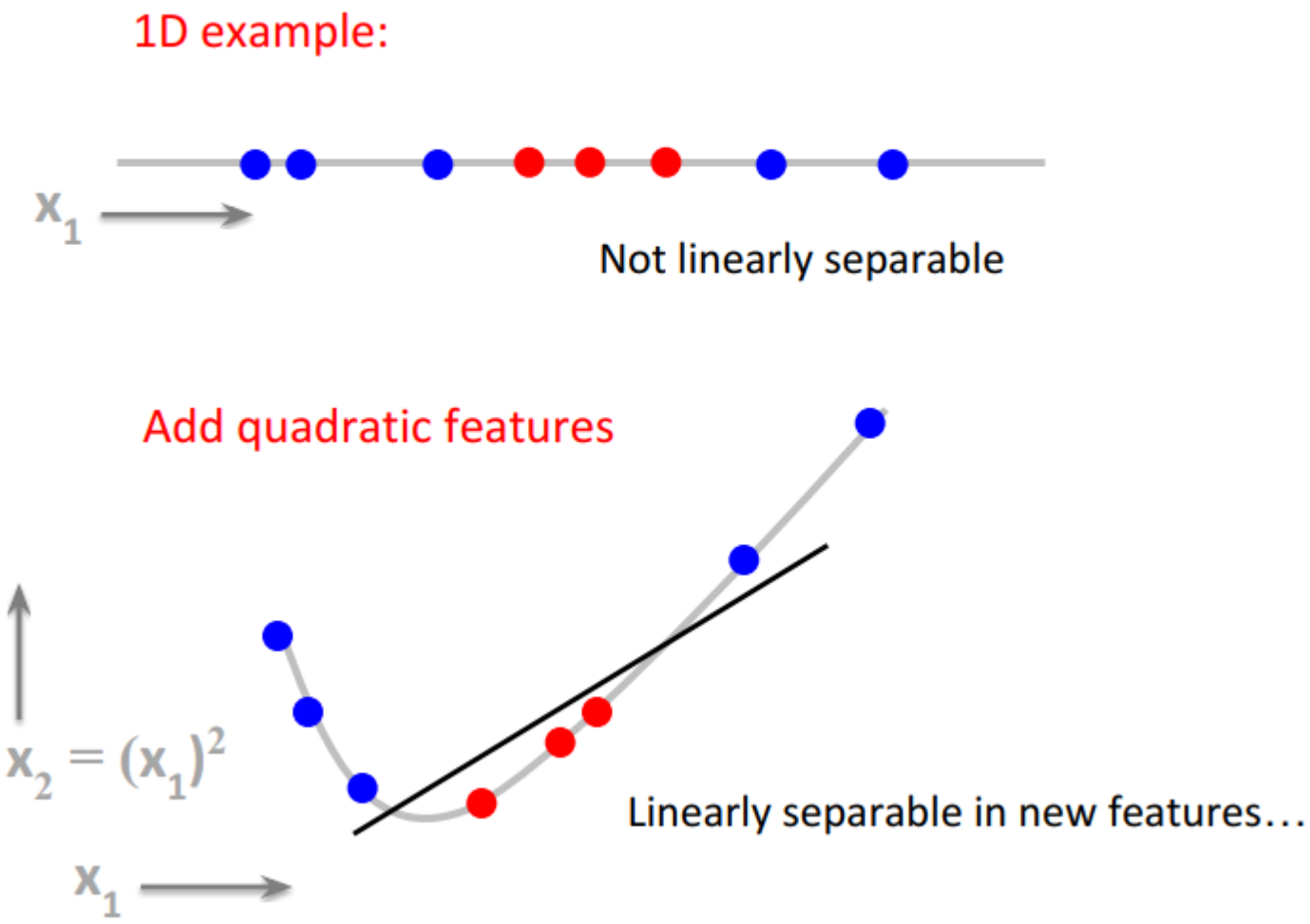
Adding Features

- Linear classifier can't learn some functions



Adding Features

- Linear classifier can't learn some functions



Adding Features

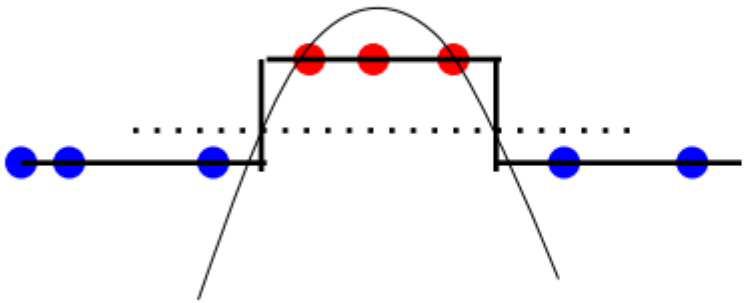
- Linear classifier can't learn some functions

1D example:

Not linearly separable

Quadratic features, visualized in original feature space:

$$y = T(a x^2 + b x + c)$$



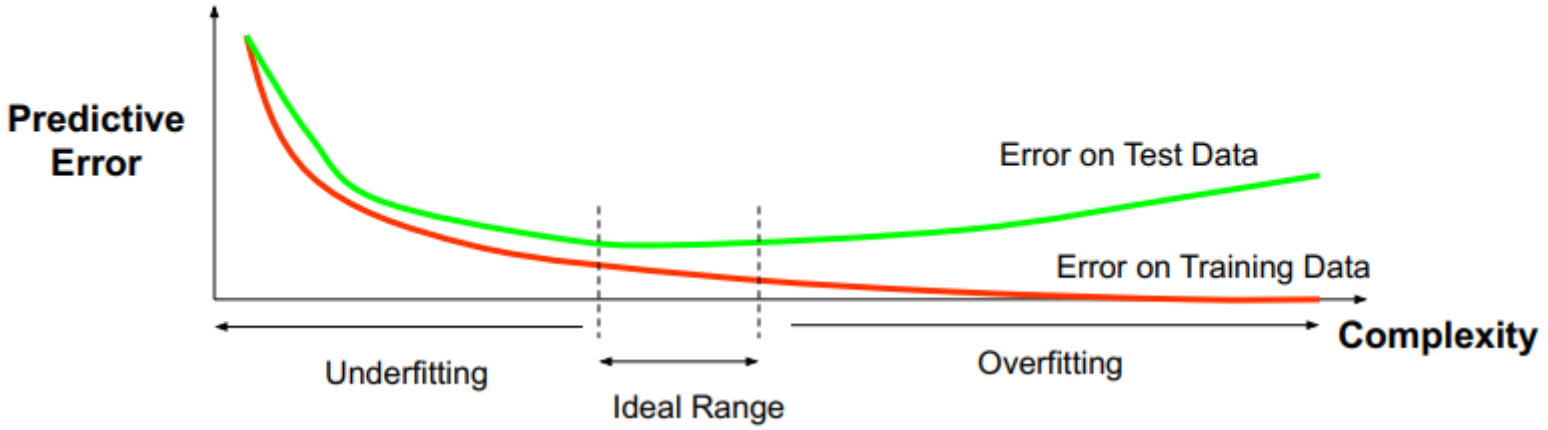
More complex decision boundary: $ax^2+bx+c = 0$

Feature Dimensionality and Model Complexity

- Data are increasingly separable in high dimension – is this a good thing?

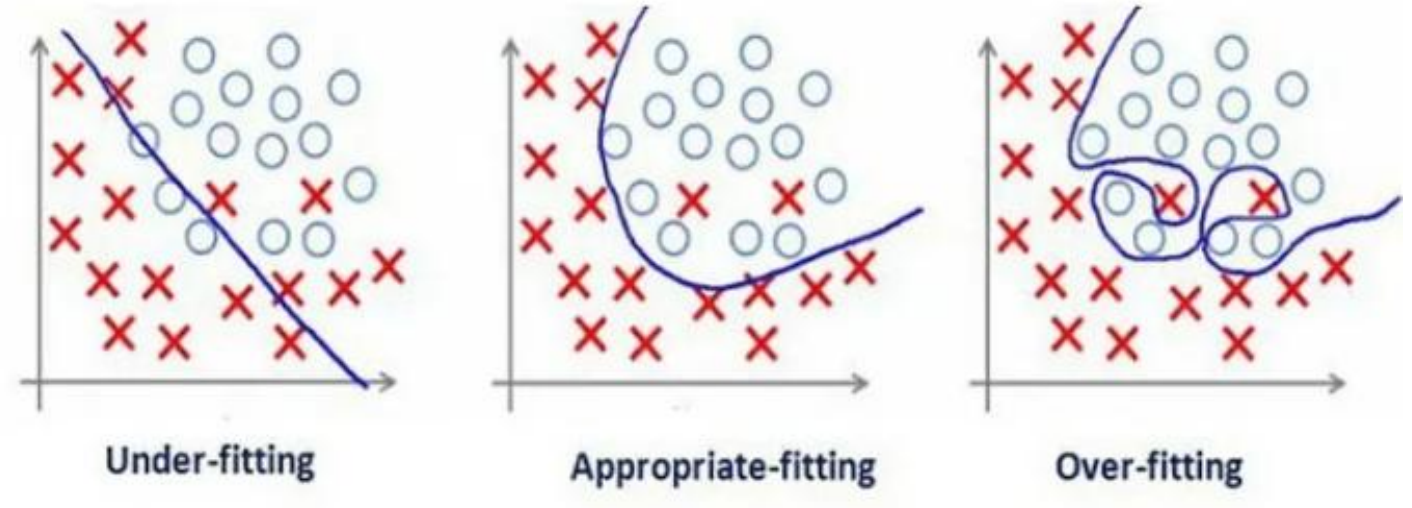
- "Good"
  - Separation is easier in higher dimensions (for fixed # of data m)
  - Increase the number of features, and even a linear classifier will eventually be able to separate all the training examples!

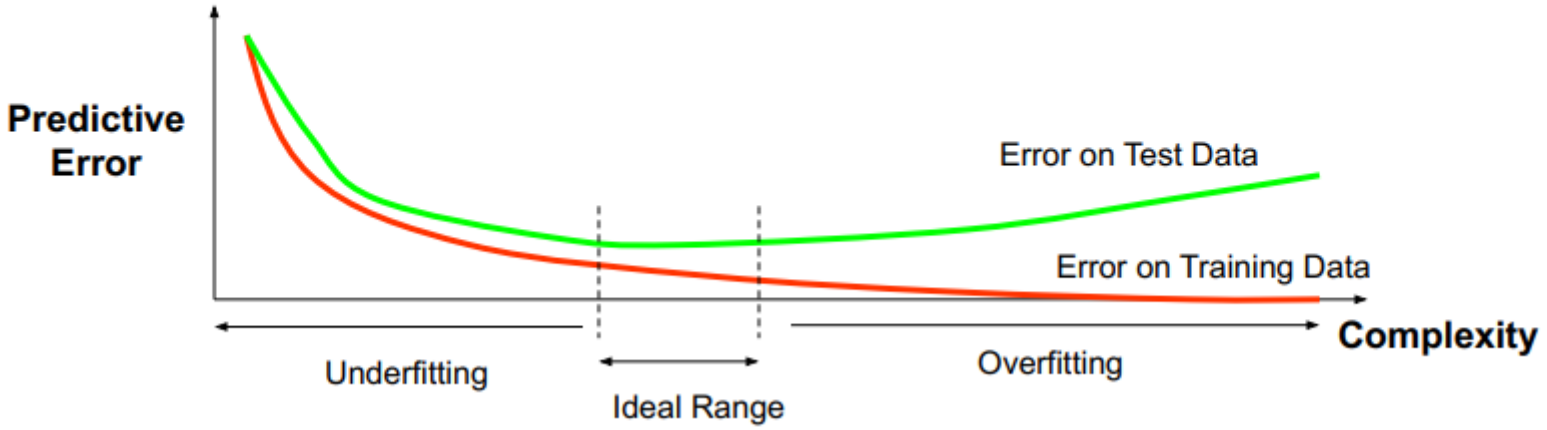
- "Bad"
  - training vs. test error? overfitting?
  - Increasingly complex decision boundaries can eventually get all the training data right, but it doesn't necessarily bode well for test data…



Feature Dimensionality and Model Complexity

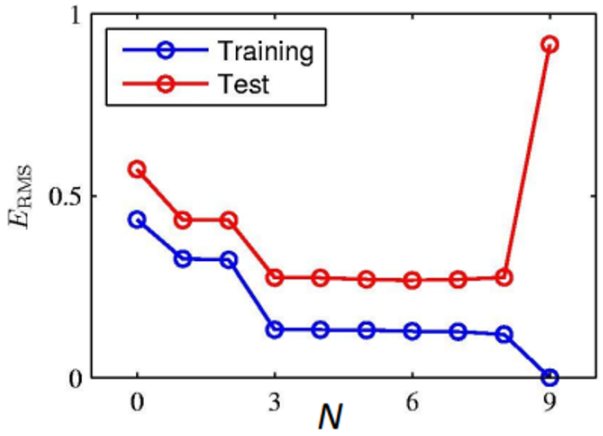
- Data are increasingly separable in high dimension – is this a good thing?





Feature Dimensionality and Model Complexity

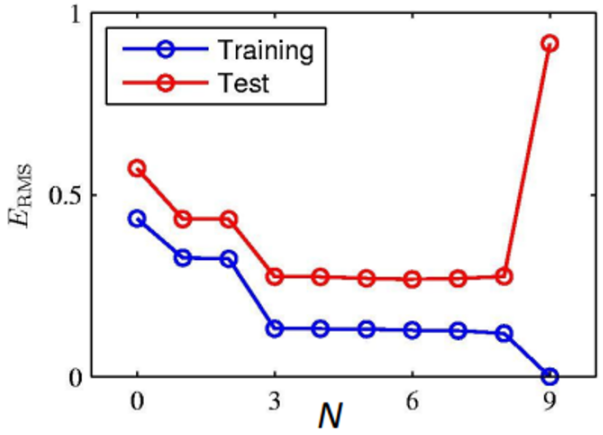
	N=0	N=1	N=3	N=9
Estimated Regression Coeffients θ	0.19	0.82	0.31	0.35
		-1.27	7.99	232.37
			-25.43	-5321.83
			17.37	48568.31
				-231639.30
				640042.26
				-1061800.52
				1042400.18
				-557682.99
				125201.43



Regularization: the L_2 Regularizer / Weight Decay

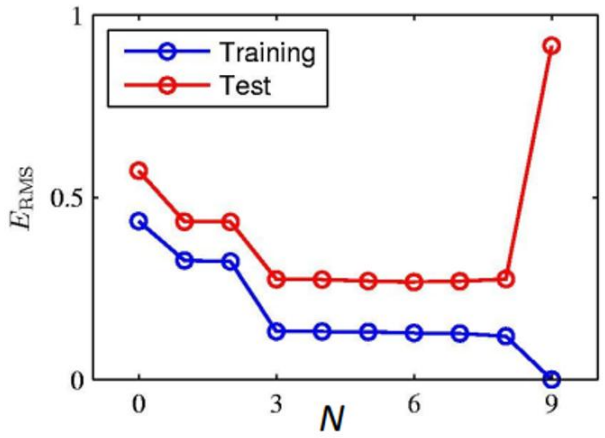
$$J(\underline{\theta}) + \alpha\,\theta\theta^T$$

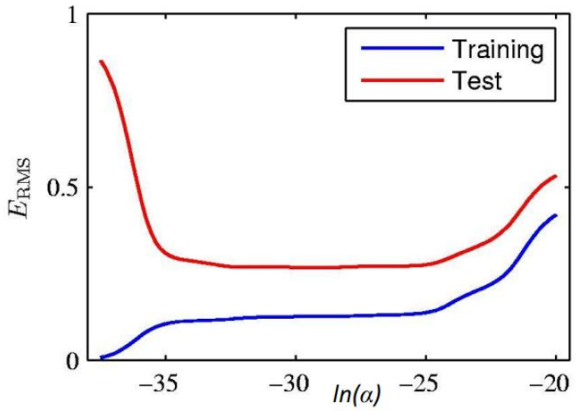
	N=0	N=1	N=3	N=9
Estimated Regression Coeffients θ	0.19	0.82	0.31	0.35
		-1.27	7.99	232.37
			-25.43	-5321.83
			17.37	48568.31
				-231639.30
				640042.26
				-1061800.52
				1042400.18
				-557682.99
				125201.43



Regularization: the L_2 Regularizer / Weight Decay

$$J(\underline{\theta}) + \alpha\,\theta\theta^T$$

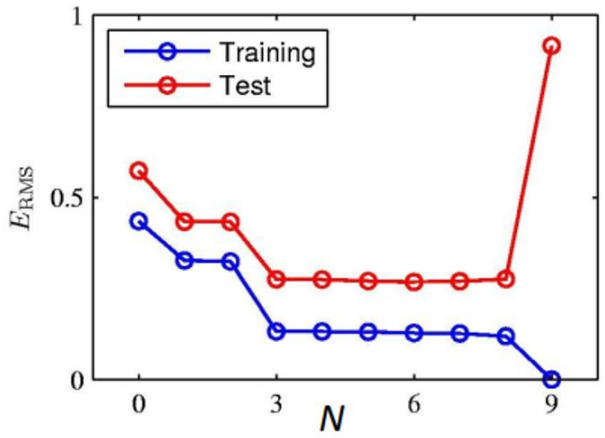


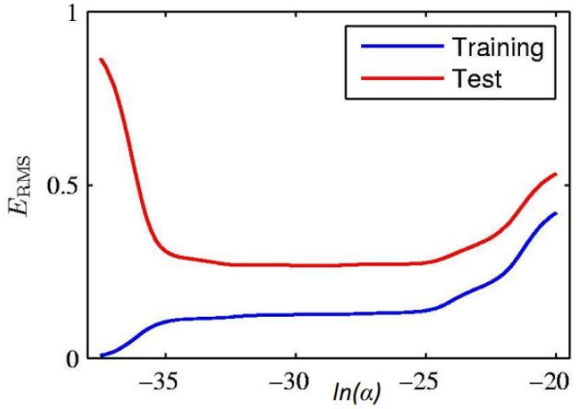


Regularization: the L_2 Regularizer / Weight Decay

$$J(\underline{\theta}) + \alpha\, \theta\theta^T$$

More generally, for the L_p regularizer: $\left(\sum_i |\theta_i|^p \right)^{\frac{1}{p}}$





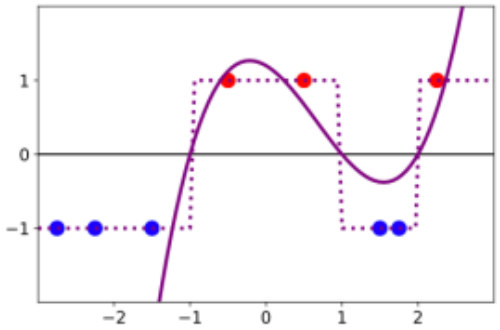
Outline

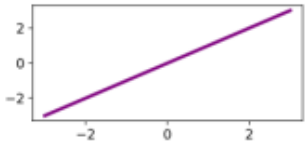
- Linear Separability
- **Multi-Layer Perceptron**
- Backpropagation
- Convolutional Neural Networks and Transformers

Features and Perceptrons

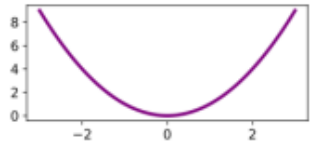
- Recall the role of features
  - Extra features can allow more complex decision boundaries
  - For example, polynomial features

$\Phi(x) = [1 \;\; x \;\; x^2 \;\; x^3 \;\; \ldots]$

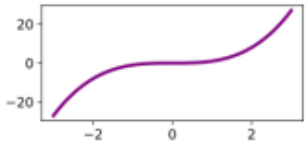




$\phi_1(x) = x$



$\phi_2(x) = x^2$

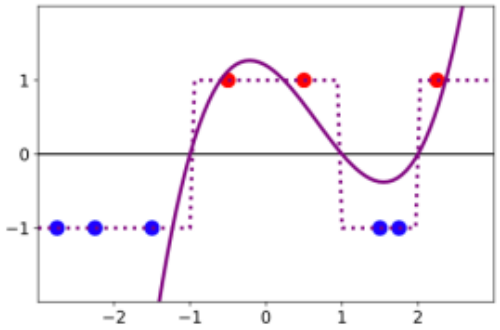


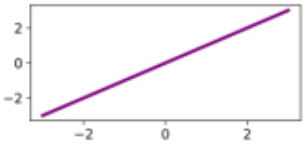
$\phi_3(x) = x^3$

$$r(x) = b + w_1\,\phi_1(x) + w_2\,\phi_2(x) + w_3\,\phi_3(x)$$

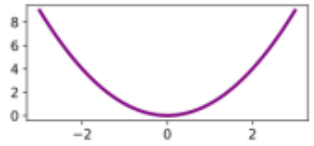
- Recall the role of features
  - Extra features can allow more complex decision boundaries
  - For example, polynomial features

$\Phi(x) = [1 \;\; x \;\; x^2 \;\; x^3 \;\; \ldots]$

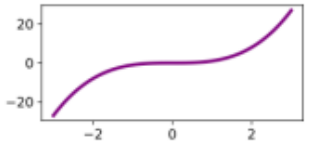




$\phi_1(x) = x$



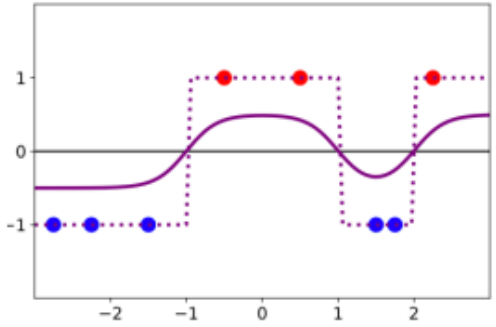
$\phi_2(x) = x^2$

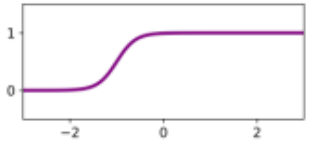


$\phi_3(x) = x^3$

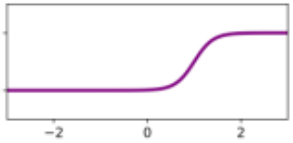
$$r(x) = b + w_1\,\phi_1(x) + w_2\,\phi_2(x) + w_3\,\phi_3(x)$$

- What other kinds of features could we choose?

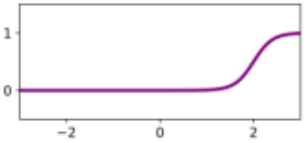




$\phi_1(x) = \sigma(5x + 5)$



$\phi_2(x) = \sigma(5x - 5)$

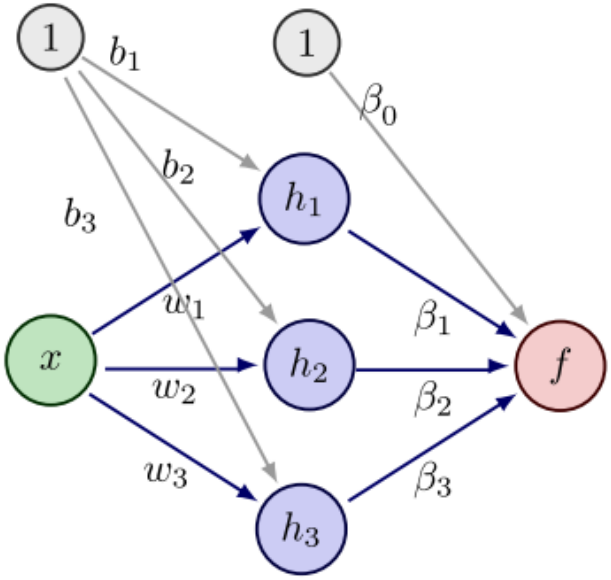


$\phi_3(x) = \sigma(5x - 10)$

$$r(x) = b + w_1\,\phi_1(x) + w_2\,\phi_2(x) + w_3\,\phi_3(x)$$

Multi-Layer Perceptron

- These features are just perceptrons!
  - Feature transform is a collection of perceptrons
  - Combination of features output of another



Logistic sigmoid:

$$\sigma(r) = \frac{1}{1+\exp(-r)}$$

$$h_1(x) = \sigma(\ b_1 + w_1 x\ )$$
$$h_2(x) = \sigma(\ b_2 + w_2 x\ )$$
$$h_3(x) = \sigma(\ b_3 + w_3 x\ )$$

$$f(x) = \sigma(\ \beta_0 + \beta_1 h_1 + \beta_2 h_2 + \beta_3 h_3\ )$$

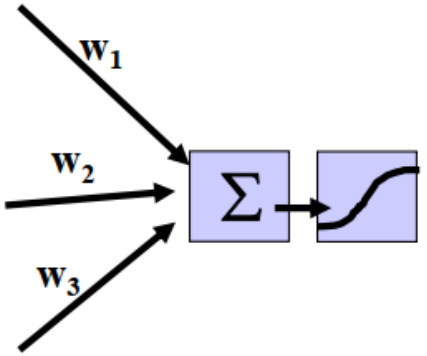
Regression version:

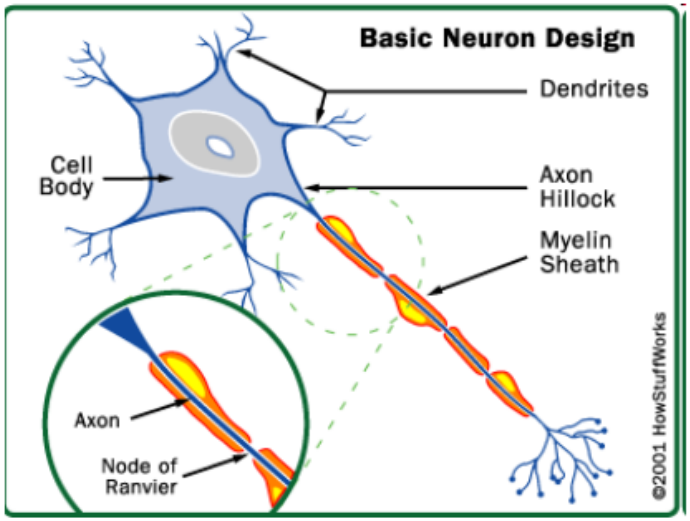
$$f(x) = \beta_0 + \beta_1 h_1 + \beta_2 h_2 + \beta_3 h_3$$

Neural Networks

- Another term for Multi-Layer Perceptrons
- Biological motivation

- Neurons
  - “Simple” cells
  - Dendrites sense charge
  - Cell weighs inputs
  - “Fires” axon

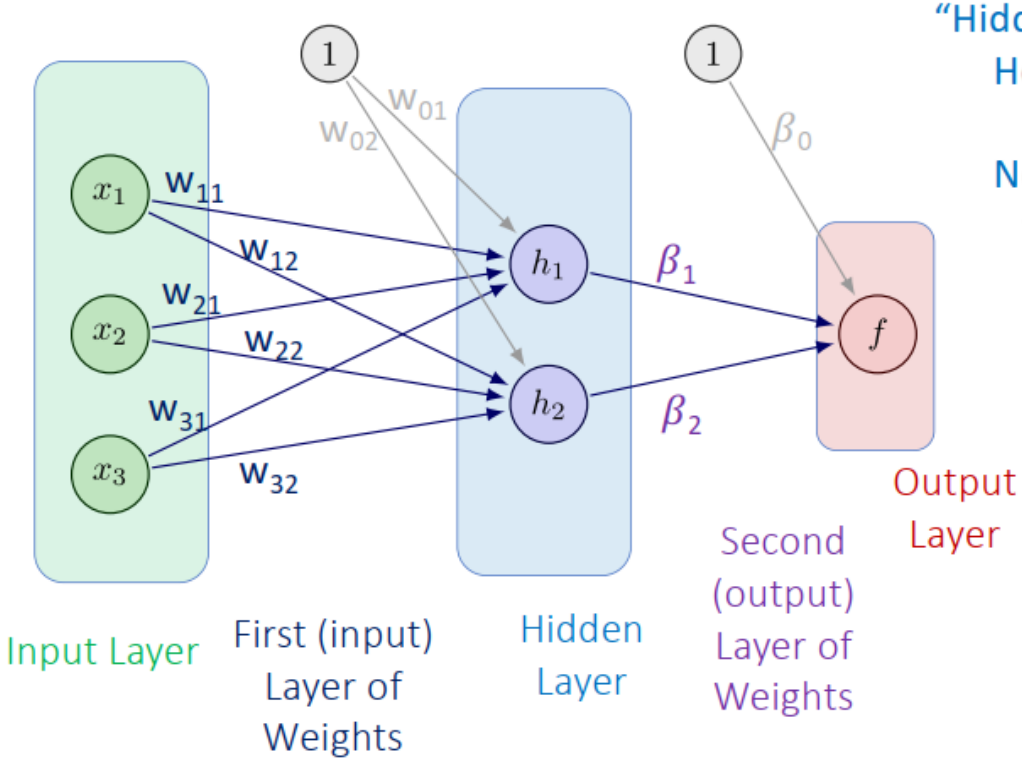




[“How stuff works: the brain”]

Two-Layer Neural Network

- "Two layer" = 1 hidden layer, 1 output layer



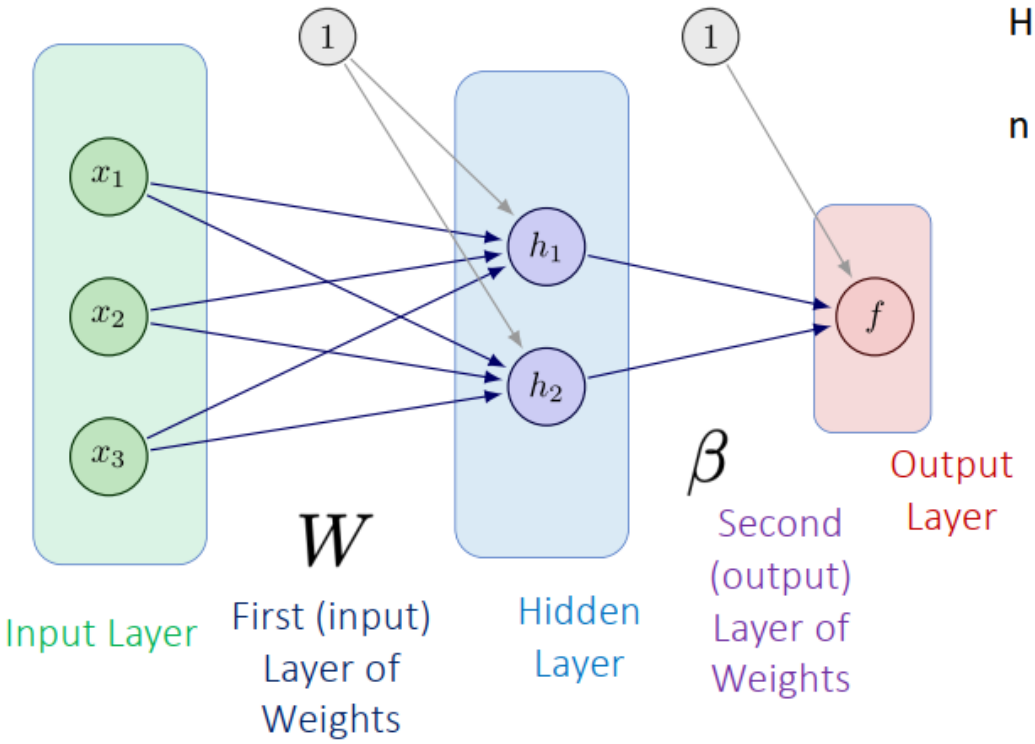
"Hidden units" = non-linear f'ns of inputs
Here, logistic sigmoid
(but other options often used)
Non-linear function is called the "activation function"

w_{ij} = weight of feature x_i in response of hidden node j

We can think of neural networks in terms of "layers" (of nodes, and of weights)

Neural Network Parameters

- Parameters θ = {all weights & biases}



How many parameters total?

n features, H hidden nodes, 1 output

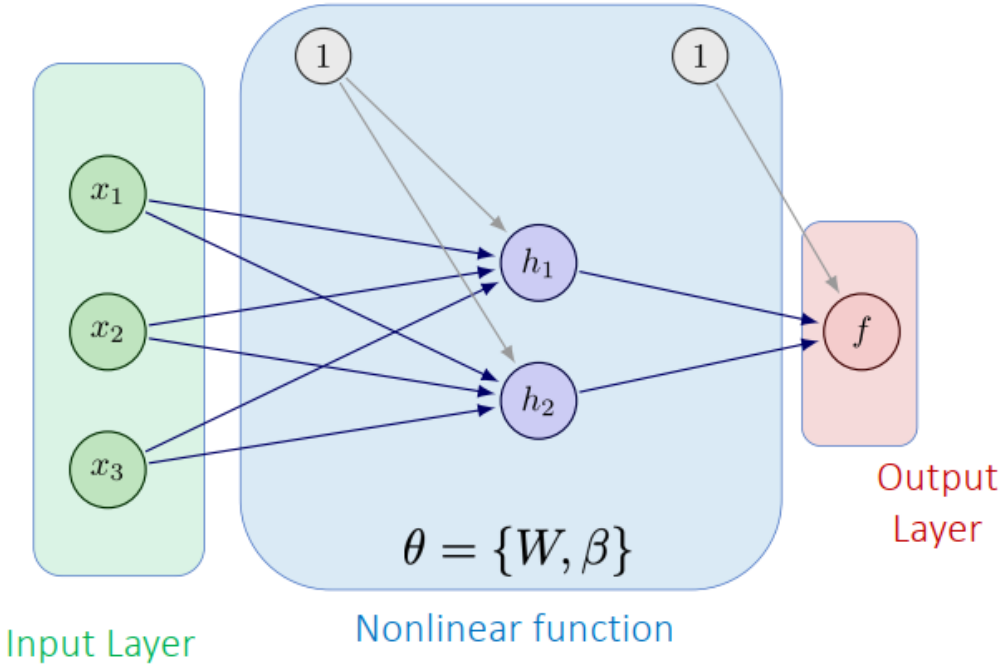
Layer 1 weights: n H
Layer 1 biases: H
Layer 2 weights: H
Layer 2 biases: 1

Total: H (n+1) + (H+1)

(approximately quadratic in layer sizes)

Neural Network Classifier

- Defines nonlinear mapping f(x ; θ)

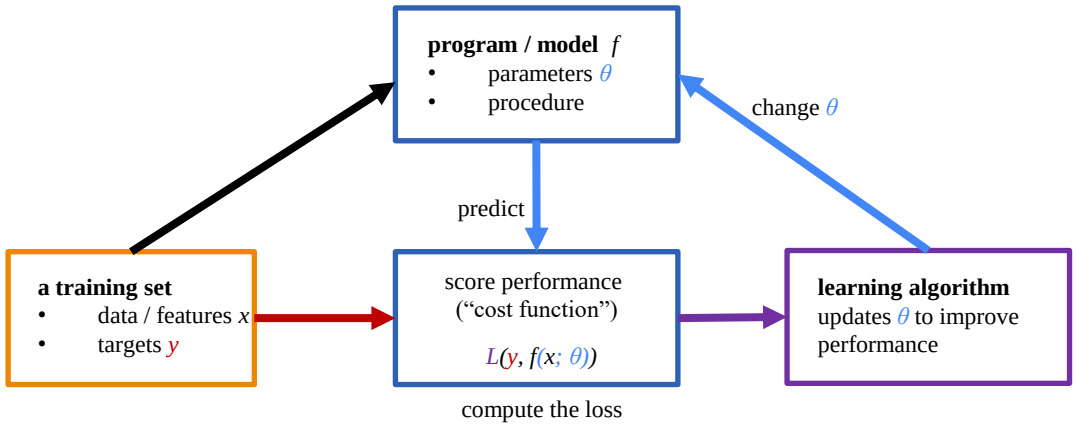


Output values in [0,1]

We can interpret these as e.g., class probabilities, p(y = 1 | x) & select log-loss (negative log-likelihood)

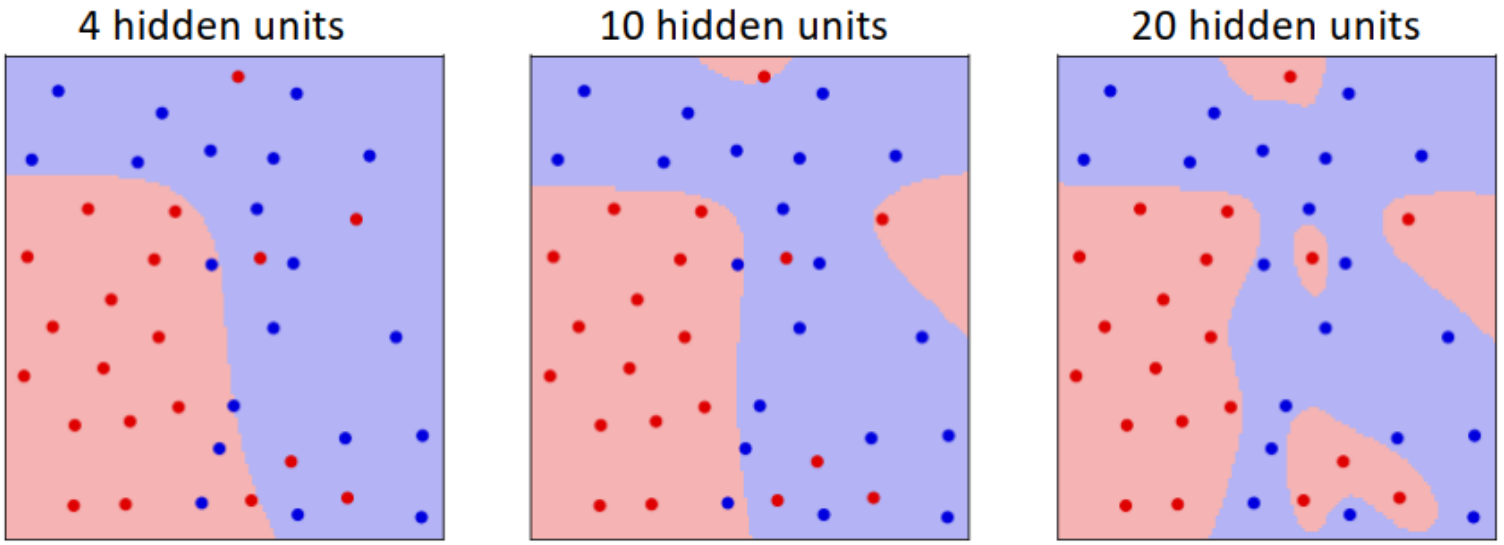
Or, we can train on MSE, $(y - f)^2$

Once we select a loss function, we can train the model!

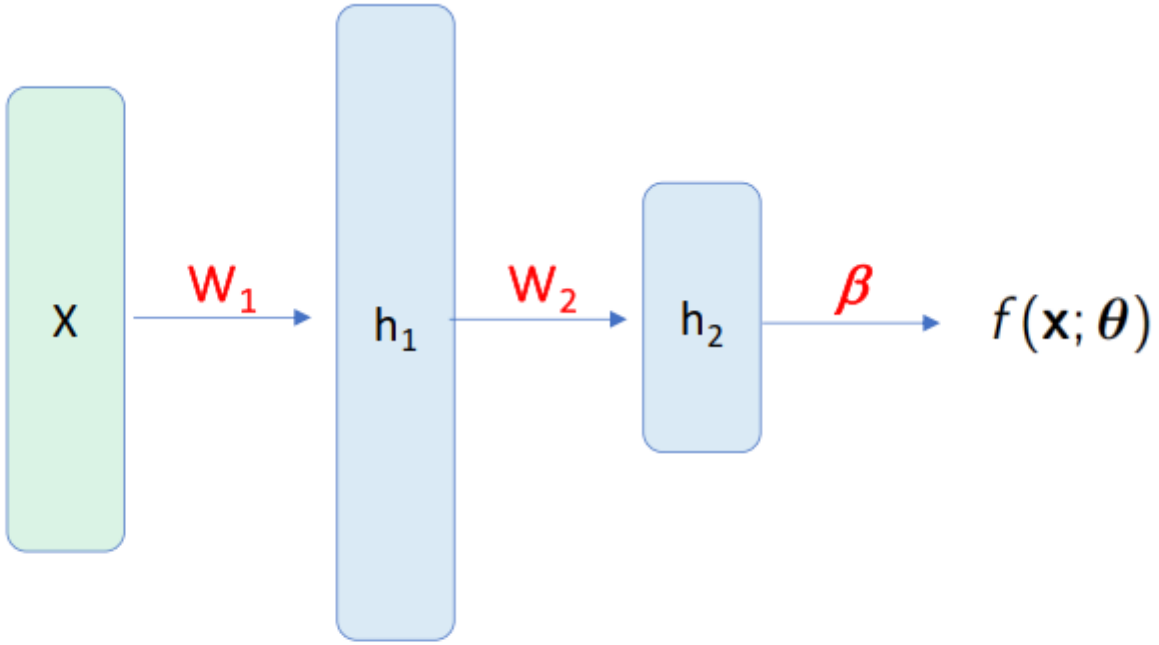


Neural Network Classifier

- Decision boundaries are non-linear
- Complexity depends on the number of layers and hidden nodes

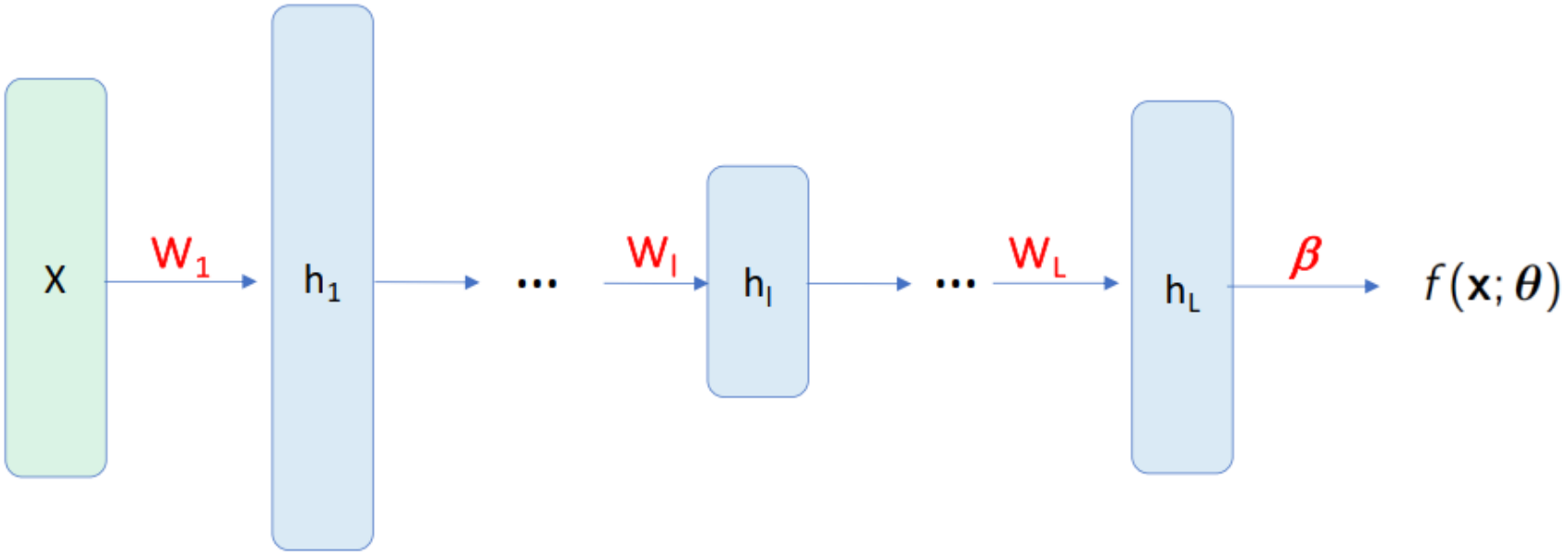


Networks with Two Hidden Layers



Each hidden unit layer can have different numbers of hidden units

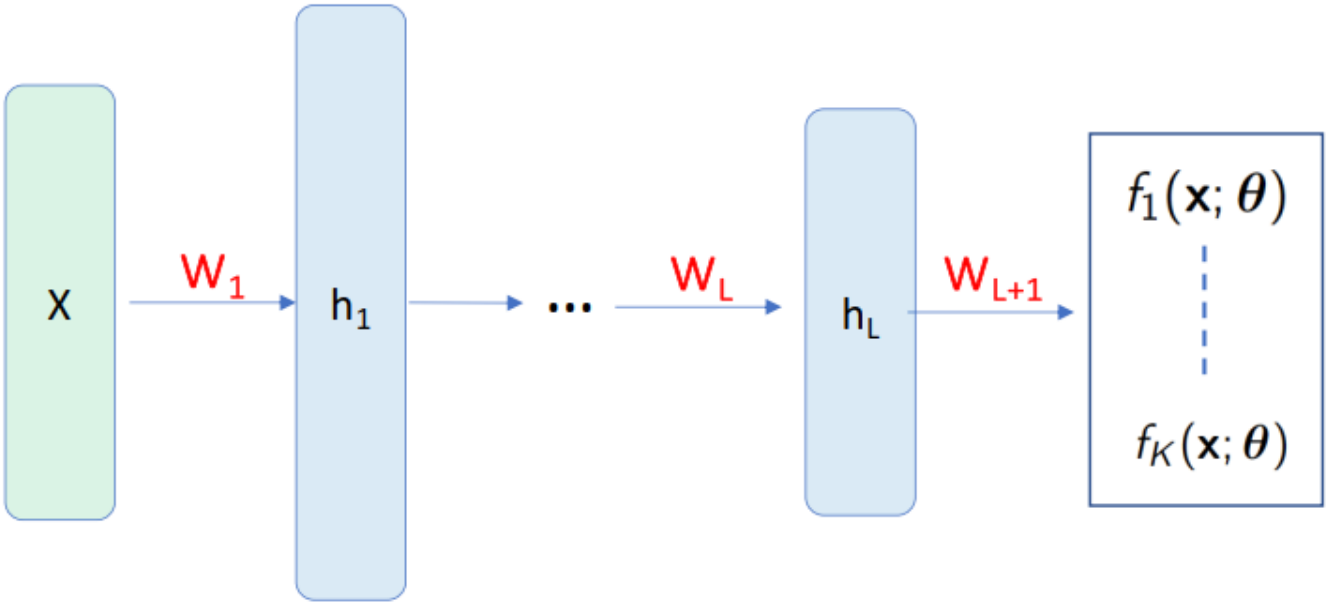
Networks with L Hidden Layers



The network can have an arbitrary number of layers, each with an arbitrary number of hidden units

Networks with multiple hidden layers are referred to as “deep”

Networks with Multiple Outputs



K different outputs, normalized by softmax function to sum to 1

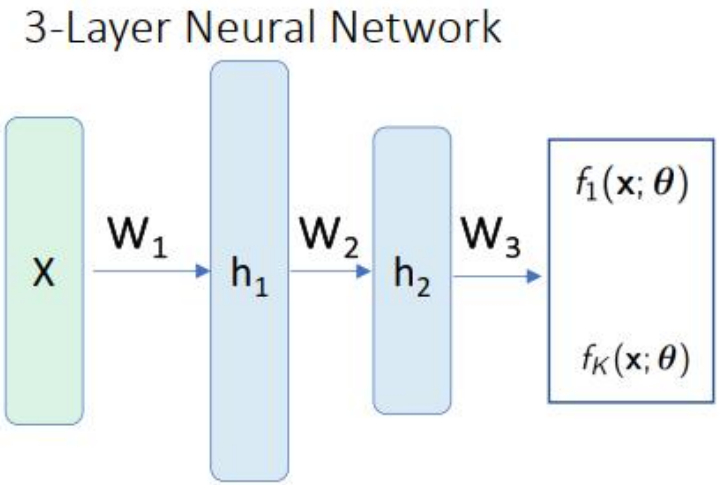
(same softmax function as for K-class logistic classifier)

$$f_k(\underline{r}) = \frac{\exp(r_k)}{\sum_{k'} \exp(r_{k'})} \qquad \left(\text{so, } \sum_k f_k = 1 \right)$$

Can interpret output c as P(y = c | x), i.e., probability of class c

(assumes only one class is correct; compare to, say, image tagging)

Example: MNIST Data

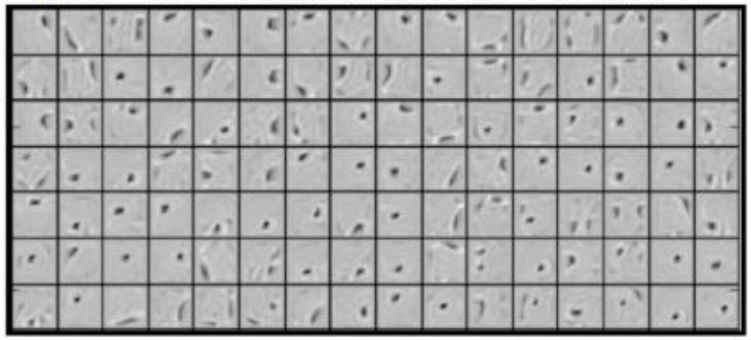


K = 10 classes 28x28 pixel resolution

784-dimensional input (pixels)

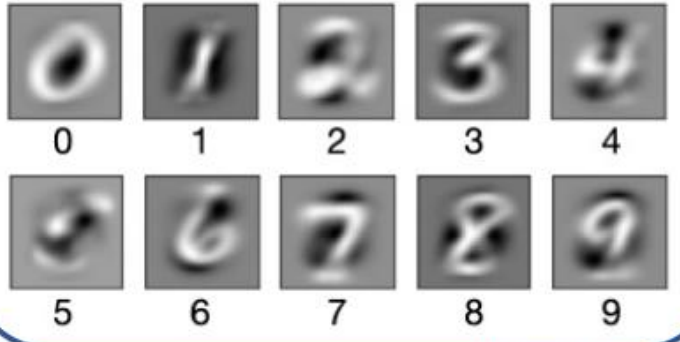
200 hidden units at each hidden layer

What do the hidden nodes learn?



(each square is a different hidden unit)

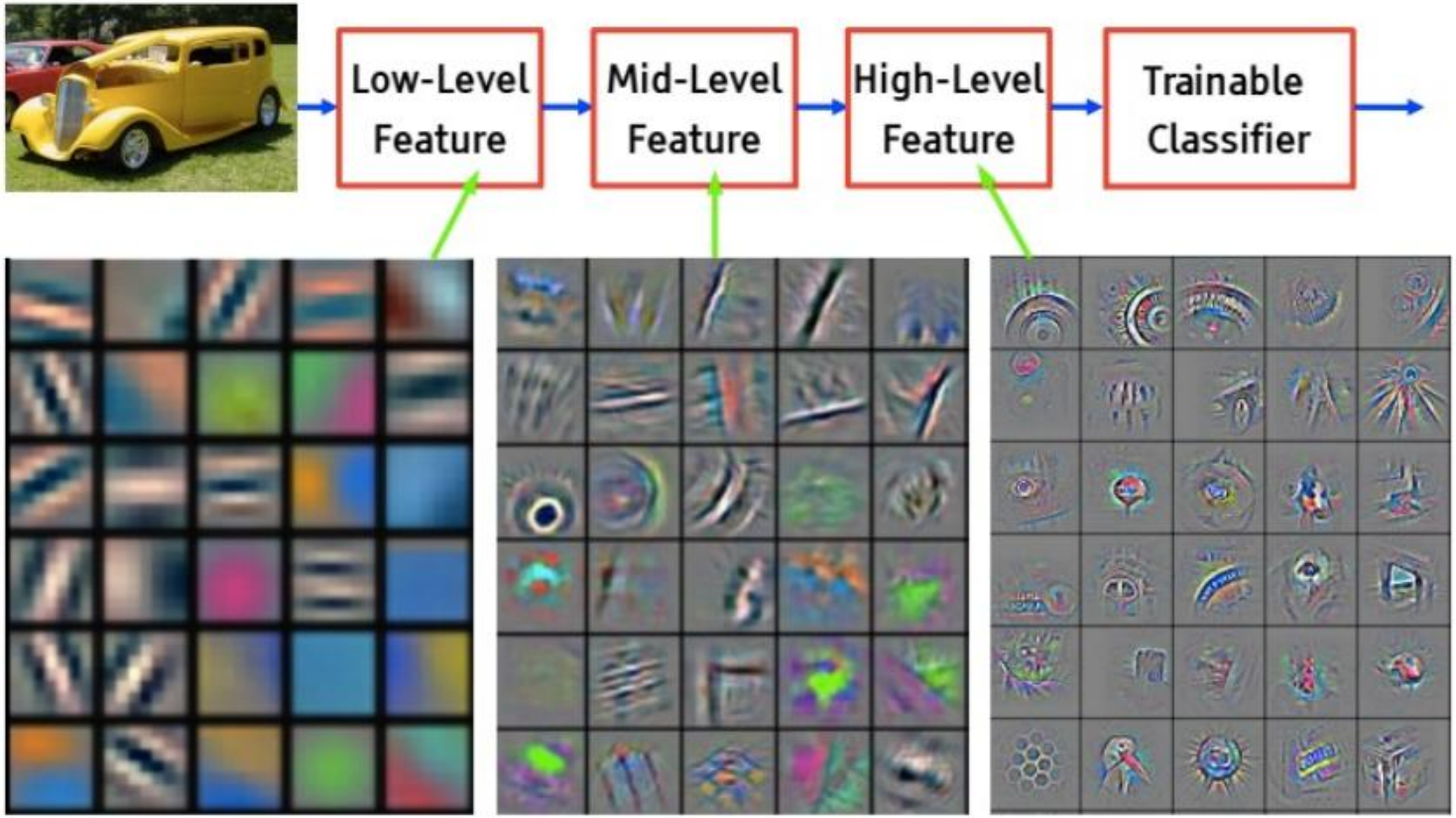
For comparison: visualizing the weights of a logistic classifier:



[Nalisnick et al., 2023]

Example: Visual Recognition

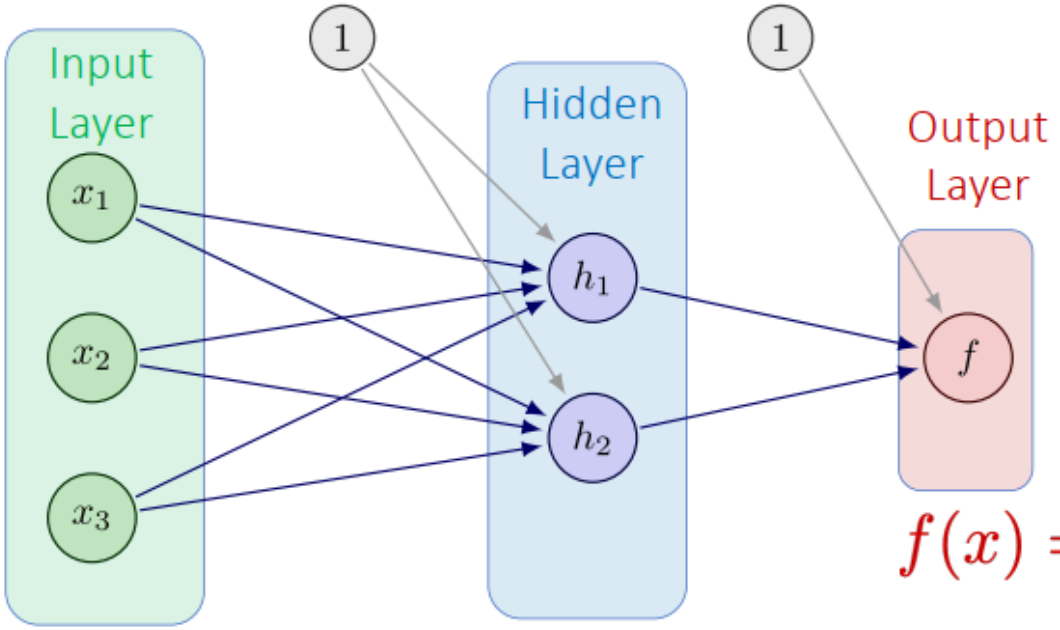
- Visualizing a convolutional network's filters [Zeiler & Fergus 2013]



Slide image from Yann LeCun:
https://drive.google.com/open?id=0BxKBnD5y2M8NclFWSXNxa0JlZTg

Activation Functions

- Each hidden node applies a nonlinearity



Logistic sigmoid:

$$\sigma(r) = \frac{1}{1+\exp(-r)}$$

$$f(x) = \sigma(\ \beta_0 + \beta_1 h_1 + \beta_2 h_2\ )$$

$$h_j(x) = \sigma(\ w_{0j} + w_{1j}\, x_1 + w_{2j}\, x_2 + w_{3j}\, x_3\ )$$

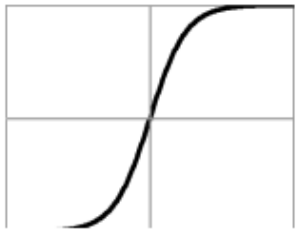
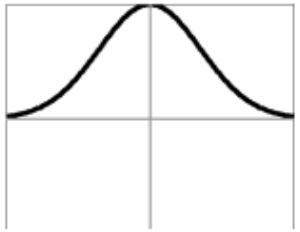
Activation Functions

Logistic

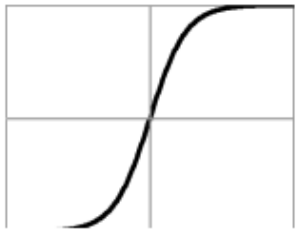
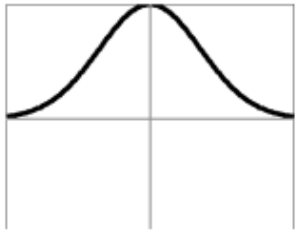
$$\sigma(z) = \frac{1}{1 + \exp(-z)}$$

$$\frac{\partial \sigma}{\partial z}(z) = \sigma(z)(1 - \sigma(z))$$

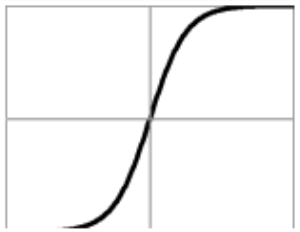
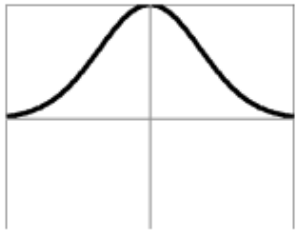
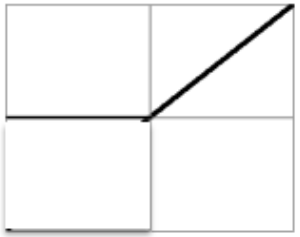
Activation Functions

Logistic	$\sigma(z) = \frac{1}{1+\exp(-z)}$		$\frac{\partial \sigma}{\partial z}(z) = \sigma(z)(1-\sigma(z))$
Hyperbolic Tangent	$\sigma(z) = \frac{1-\exp(-2z)}{1+\exp(-2z)}$		$\frac{\partial \sigma}{\partial z}(z) = 1-(\sigma(z))^2$
Gaussian	$\sigma(z) = \exp(-z^2/2)$		$\frac{\partial \sigma}{\partial z}(z) = -z\sigma(z)$

Activation Functions

Logistic	$\sigma(z) = \frac{1}{1+\exp(-z)}$		$\frac{\partial \sigma}{\partial z}(z) = \sigma(z)(1-\sigma(z))$
Hyperbolic Tangent	$\sigma(z) = \frac{1-\exp(-2z)}{1+\exp(-2z)}$		$\frac{\partial \sigma}{\partial z}(z) = 1-(\sigma(z))^2$
Gaussian	$\sigma(z) = \exp(-z^2/2)$		$\frac{\partial \sigma}{\partial z}(z) = -z\sigma(z)$

Activation Functions

Logistic	$\sigma(z) = \frac{1}{1+\exp(-z)}$		$\frac{\partial \sigma}{\partial z}(z) = \sigma(z)(1-\sigma(z))$
Hyperbolic Tangent	$\sigma(z) = \frac{1-\exp(-2z)}{1+\exp(-2z)}$		$\frac{\partial \sigma}{\partial z}(z) = 1-(\sigma(z))^2$
Gaussian	$\sigma(z) = \exp(-z^2/2)$		$\frac{\partial \sigma}{\partial z}(z) = -z\sigma(z)$
ReLU (rectified linear)	$\sigma(z) = \max(0, z)$		$\frac{\partial \sigma}{\partial z}(z) = \mathbb{1}[z > 0]$

Activation Functions

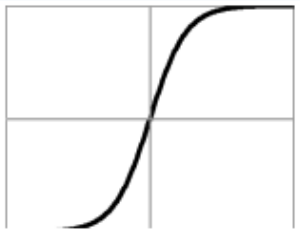
Logistic

$$\sigma(z) = \frac{1}{1+\exp(-z)}$$

$$\frac{\partial \sigma}{\partial z}(z) = \sigma(z)(1-\sigma(z))$$

Hyperbolic Tangent

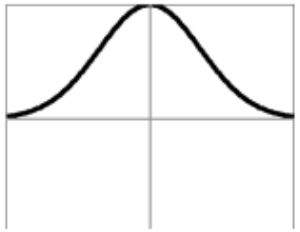
$$\sigma(z) = \frac{1-\exp(-2z)}{1+\exp(-2z)}$$



$$\frac{\partial \sigma}{\partial z}(z) = 1-(\sigma(z))^2$$

Gaussian

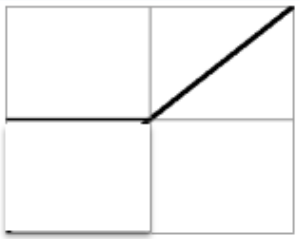
$$\sigma(z) = \exp(-z^2/2)$$



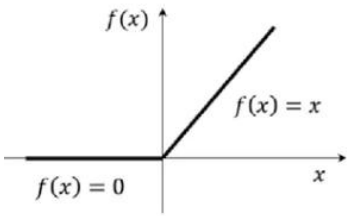
$$\frac{\partial \sigma}{\partial z}(z) = -z\sigma(z)$$

ReLU (rectified linear)

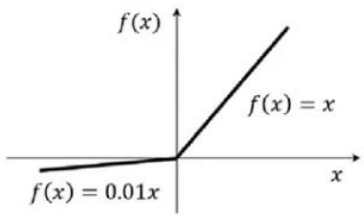
$$\sigma(z) = \max(0, z)$$



$$\frac{\partial \sigma}{\partial z}(z) = \mathbb{1}[z > 0]$$



ReLU activation function



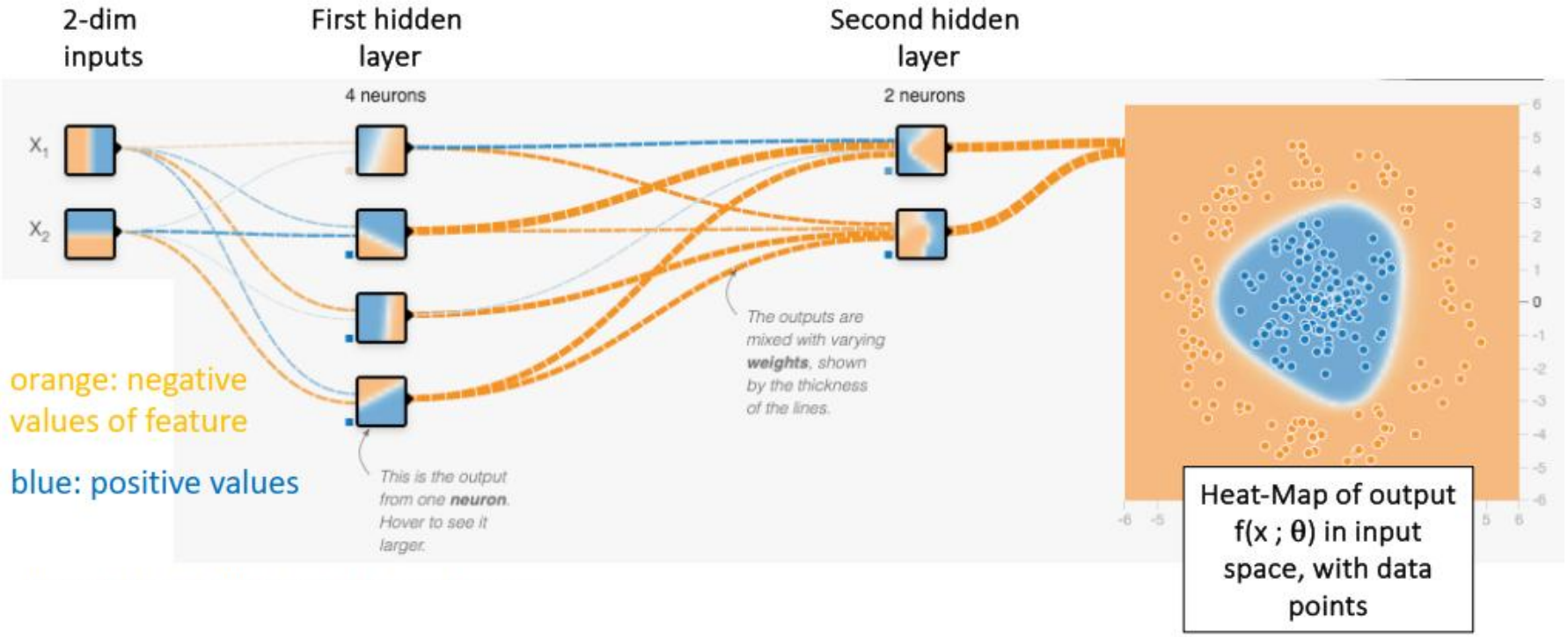
LeakyReLU activation function

Example: Simple Network with 2 Hidden Layers

http://playground.tensorflow.org/

Two layers

- Layer 1: 4 hidden nodes
- Layer 2: 2 hidden nodes
- Activation: tanh

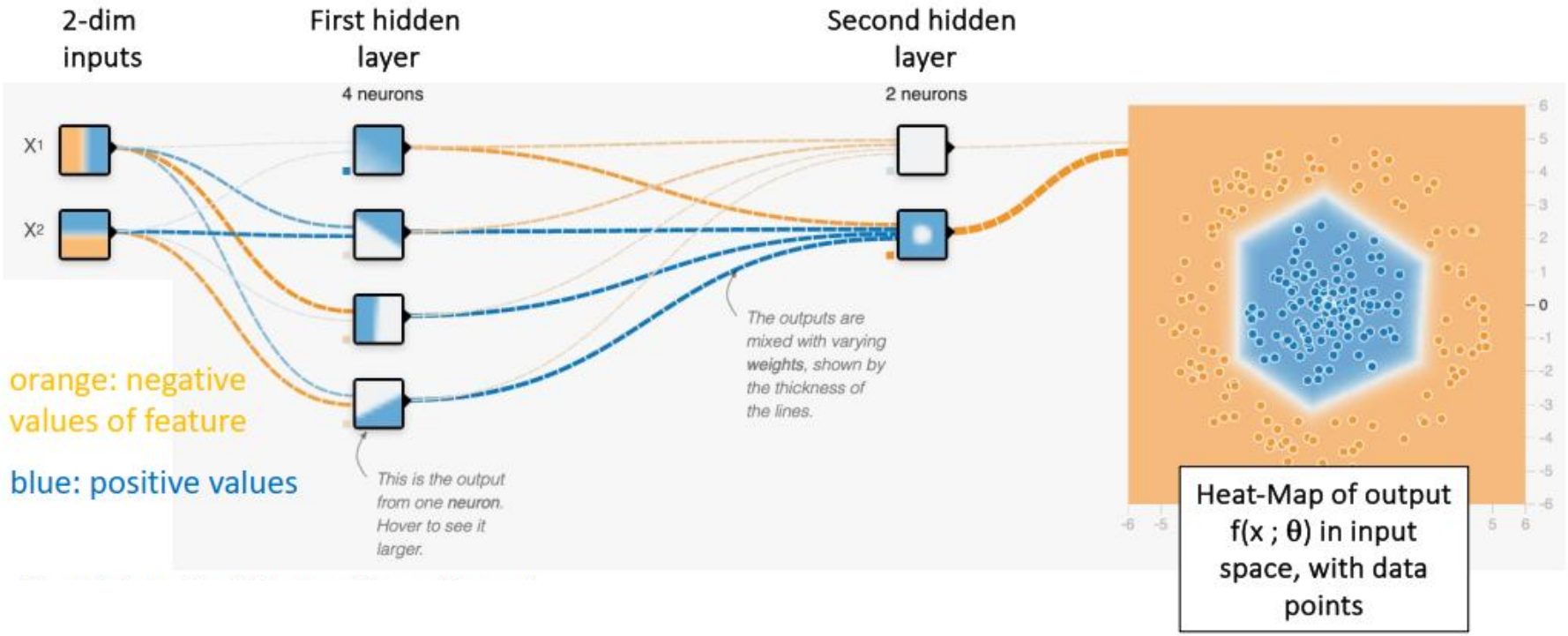


Example: Simple Network with 2 Hidden Layers

http://playground.tensorflow.org/

For comparison: same data, ReLU nonlinearity

- Now features are piecewise linear functions
- So, decision boundary is also piecewise linear
  - # pieces depends on # of layers and nodes…



Outline

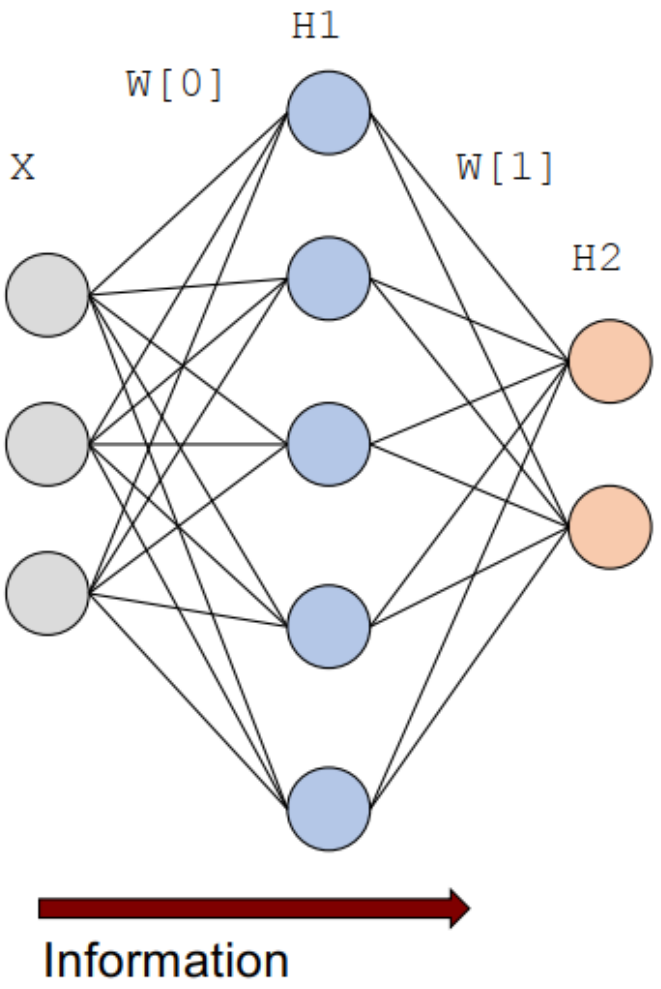
- Linear Separability
- Multi-Layer Perceptron
- **Backpropagation**
- Convolutional Neural Networks and Transformers

Feed-Forward Networks

- Information flows left-to-right
  - Input observed features
  - Compute hidden nodes (parallel)
  - Compute next layer…

```
R = X @ W[0] + B[0]   # linear response
H1= Act( R )          # activation f'n

S = H1 @ W[1] + B[1] # linear response
H2 = Act( S )         # activation f'n
```

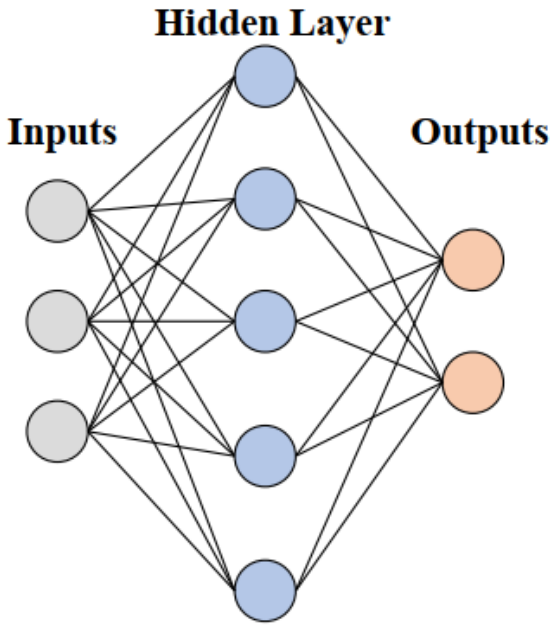


Training MLPs

- Observe features “x” with target “y”
- Push “x” through NN = output is “f”
- Error: $(y - f)^2$ (Can use different loss functions if desired; e.g., log-loss/NLL)
- How should we update the weights to improve?

- Single layer
  - Logistic sigmoid function
  - Smooth, differentiable
- Optimize using:

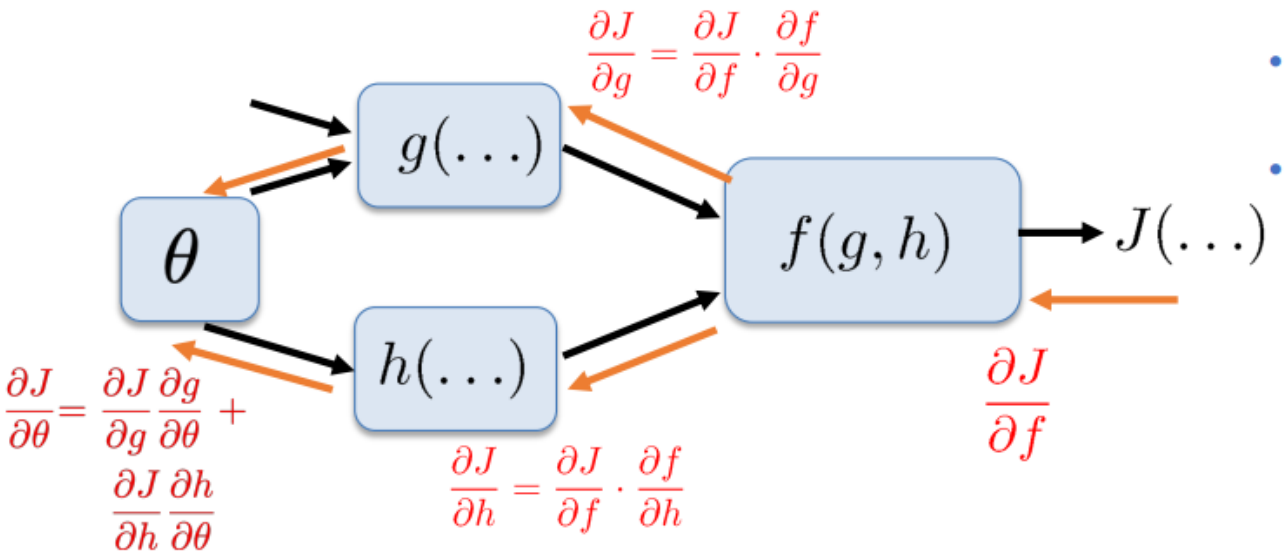
  Gradient Descent along the chains

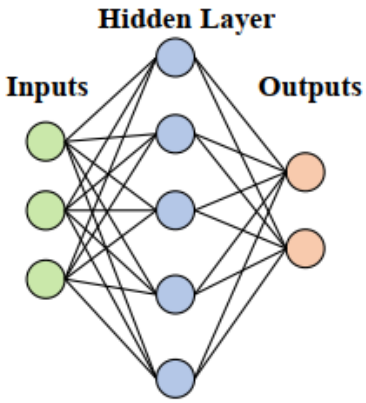


Gradient Calculations

- Think of NNs as "schematics" made of smaller functions
  - Building blocks: summations & nonlinearities
  - For derivatives, just apply the chain rule, etc!

Backward pass:
- sum incoming derivative messages
- multiply by own slope wrt each argument
- Send back to each arg





save & reuse info (g,h) from forward computation!

Ex: $f(g,h) = g^2 h$

$$\frac{\partial J}{\partial g} = \frac{\partial J}{\partial f} \cdot 2\, g(\cdot)\, h(\cdot) \qquad \frac{\partial J}{\partial h} = \frac{\partial J}{\partial f} \cdot g^2(\cdot)$$

Backpropagation

- Just gradient descent…
- Apply the chain rule to the MLP

Forward pass

Loss function

$$J_i(W) = \sum_k (y_k^{(i)} - \hat{y}_k^{(i)})^2$$

Output layer

$$\hat{y}_k = \sigma(s_k) = \sigma(\sum_j w_{kj}^2 h_j)$$

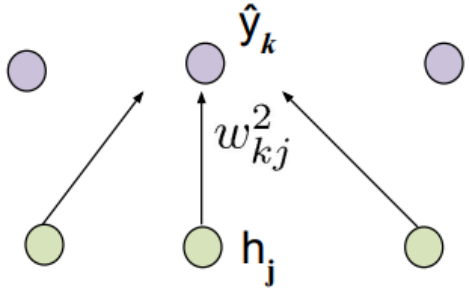
Hidden layer

$$h_j = \sigma(t_j) = \sigma(\sum_i w_{ji}^1 x_i)$$

$$\frac{\partial J}{\partial w_{kj}^2} = -2 \sum_{k'} (y_{k'} - \hat{y}_{k'}) \; (\partial \hat{y}_{k'})$$

$$= -2(y_k - \hat{y}_k) \; \sigma'(s_k) \; h_j$$

(Identical to logistic mse regression with inputs "h_j")



Backpropagation

- Just gradient descent…
- Apply the chain rule to the MLP

Forward pass

Loss function

$$J_i(W) = \sum_k (y_k^{(i)} - \hat{y}_k^{(i)})^2$$

Output layer

$$\hat{y}_k = \sigma(s_k) = \sigma(\sum_j w_{kj}^2 h_j)$$

Hidden layer

$$h_j = \sigma(t_j) = \sigma(\sum_i w_{ji}^1 x_i)$$

$$\frac{\partial J}{\partial w_{kj}^2} = -2 \sum_{k'} (y_{k'} - \hat{y}_{k'}) \; (\partial \hat{y}_{k'})$$

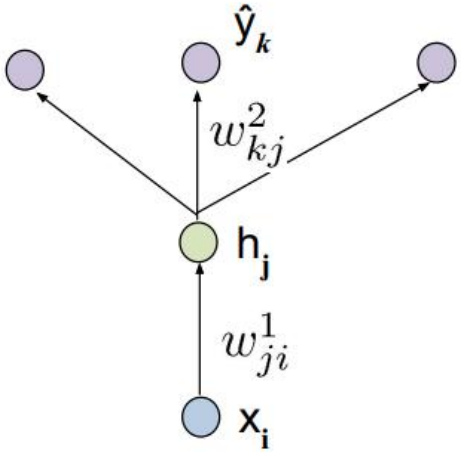
$$= \boxed{-2(y_k - \hat{y}_k) \; \sigma'(s_k)} \; h_j$$

(Identical to logistic mse regression with inputs "h_j")

$$\frac{\partial J}{\partial w_{ji}^1} = \sum_k -2(y_k - \hat{y}_k) \; (\partial \hat{y}_k)$$

$$= \sum_k -2(y_k - \hat{y}_k) \; \sigma'(s_k) \; w_{kj}^2 \; \partial h_j$$

$$= \sum_k \boxed{-2(y_k - \hat{y}_k) \; \sigma'(s_k)} \; w_{kj}^2 \; \sigma'(t_j) \; x_i$$



Outline

- Linear Separability
- Multi-Layer Perceptron
- Backpropagation
- **Convolutional Neural Networks and Transformers**

A Little History on Neural Networks

- Phase 1, 1950s to 1970s
  - Logistic-like models, no hidden units
  - Initial enthusiasm died out
- Phase 2, 1980s to 2000s
  - Invention of backpropagation: could train models with hidden units
  - But training was slow, data was scarce….initial enthusiasm died out
- Phase 3, 2010s to present
  - Demonstrations of the power of deep learning models
  - (re)invention of a technique called stochastic gradient
  - Commercial successes, great enthusiasm….

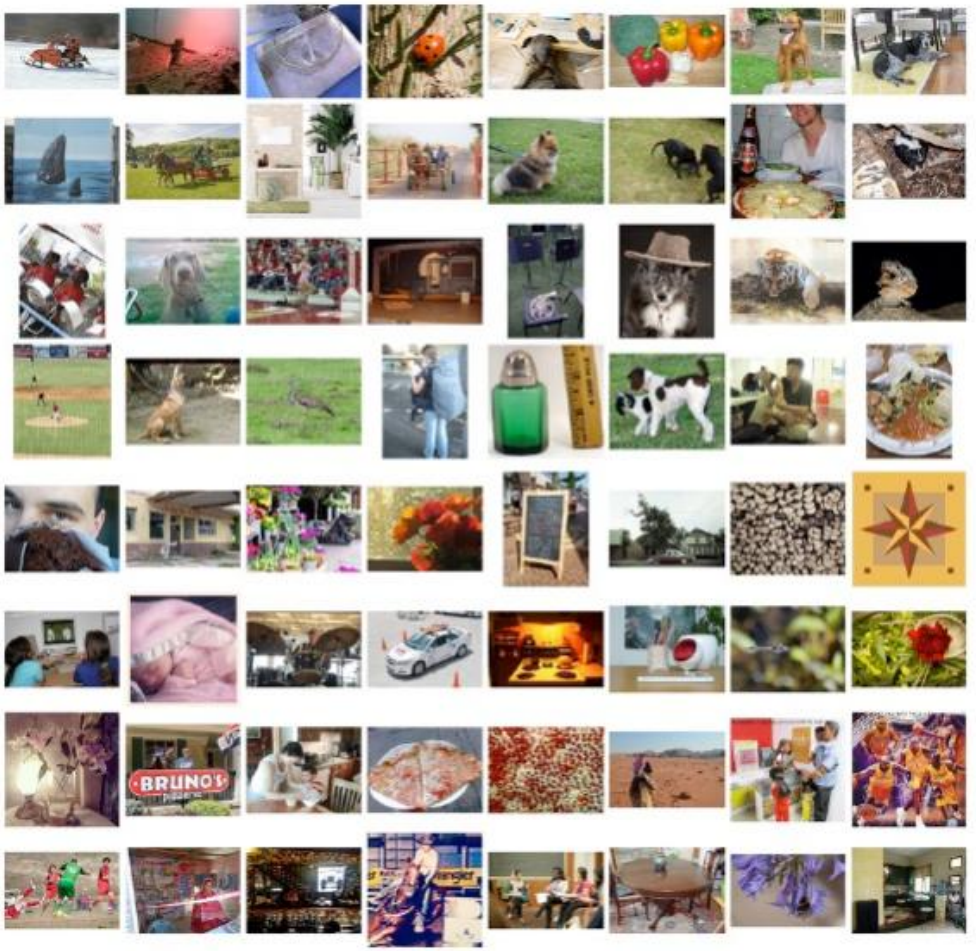
Application: Image Recognition

ImageNet

A testbed for evaluating image classification algorithms

Over 10 million images

1000 class labels



From Russakovsky et al., ImageNet Large Scale Visual Recognition Challenge, 2015

Application: Image Recognition

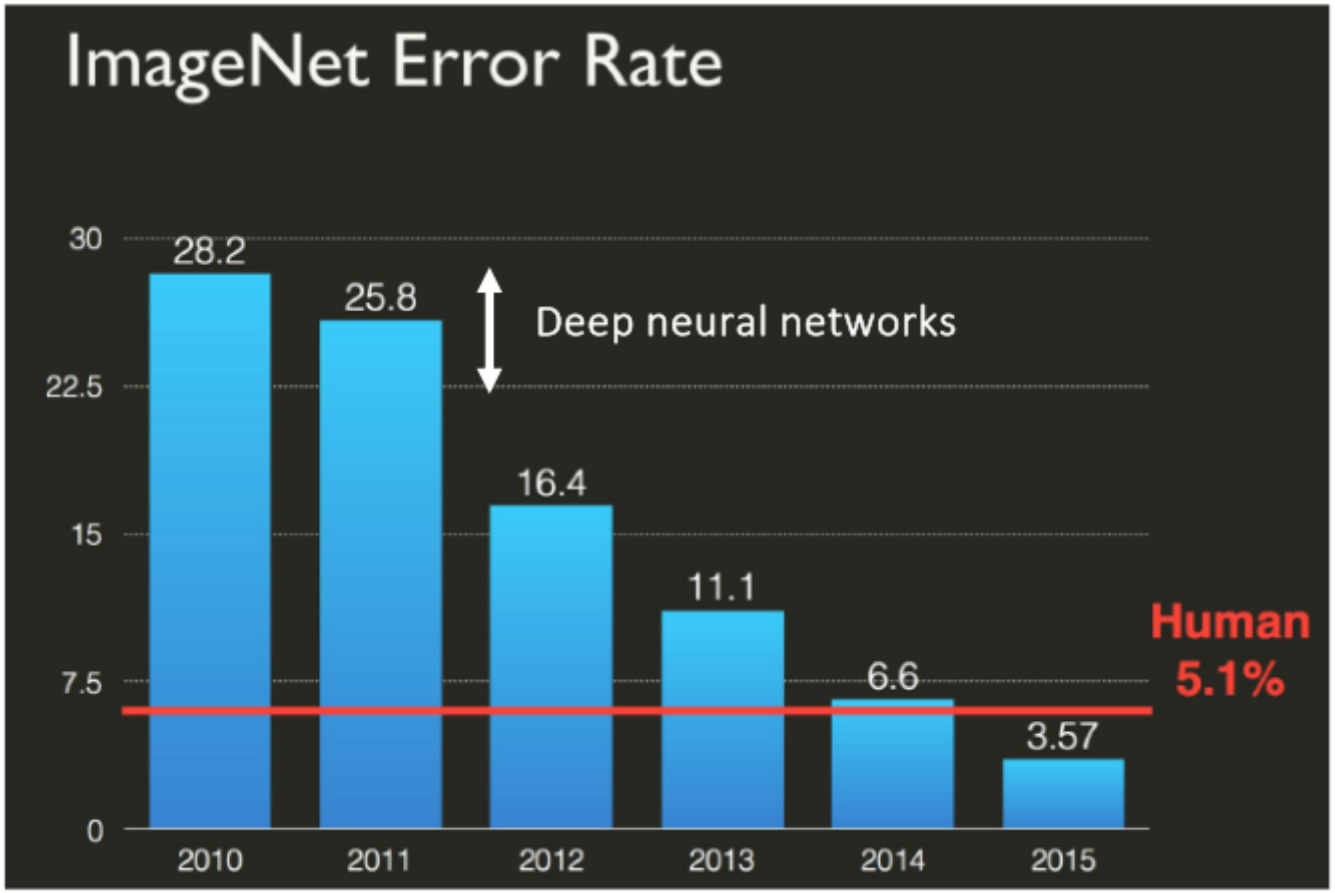
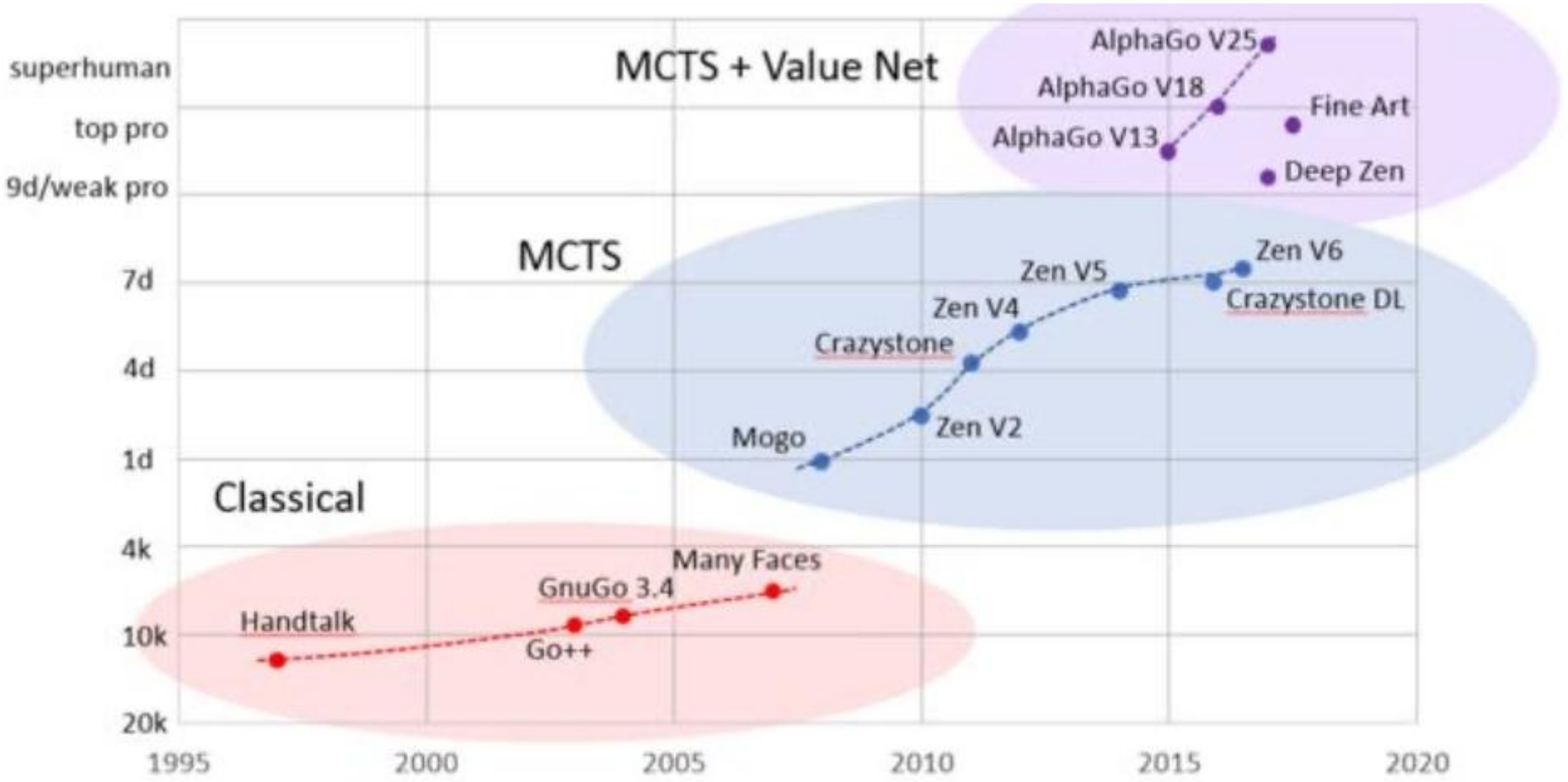


Figure from Kevin Murphy, Google

Application: Go



https://www.reddit.com/r/baduk/comments/6ttyyz/better_graph_of_go_ai_strength_over_time/

Deep learning basics – image classification

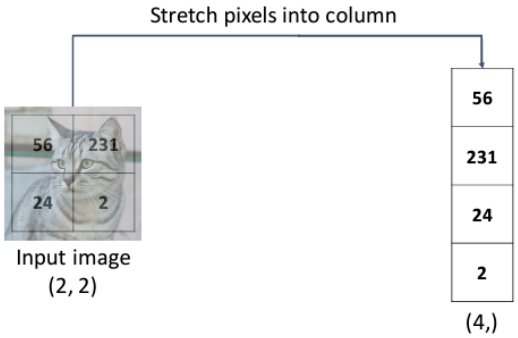
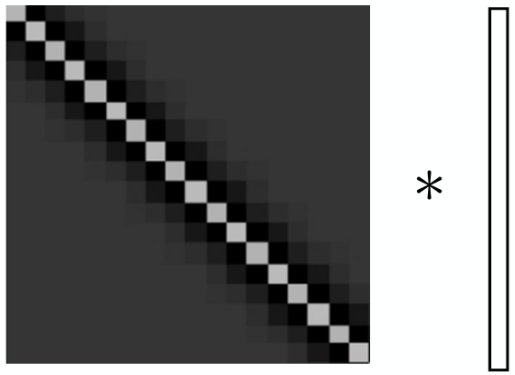


image **x**



label y

Deep learning basics – image classification with more layers of weights

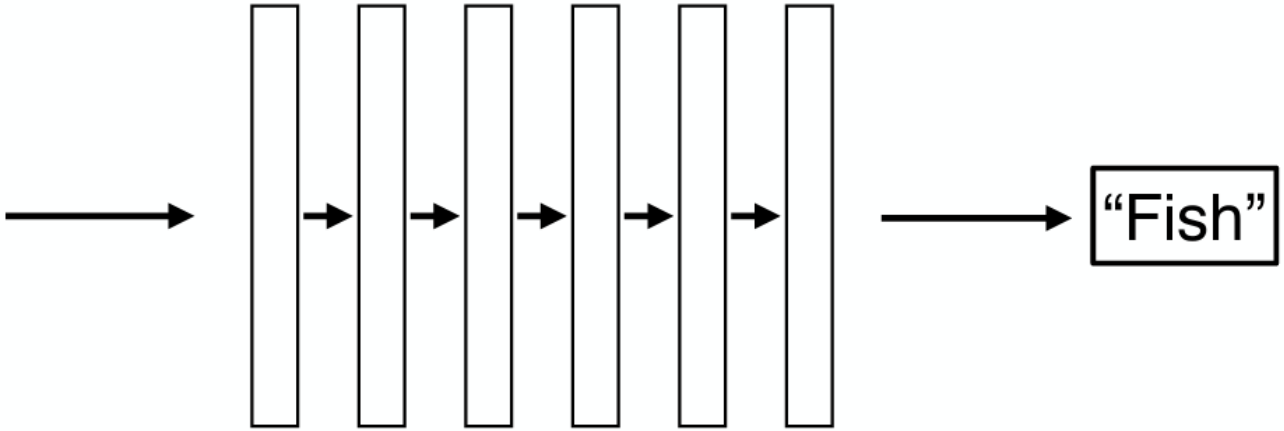


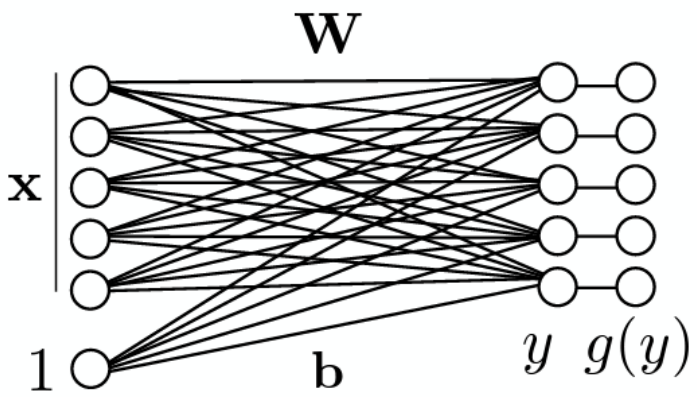
image **x**

label y

$$\mathbf{x}^{(l+1)} = \mathbf{W}^{(l)} * \mathbf{x}^{(l)}$$

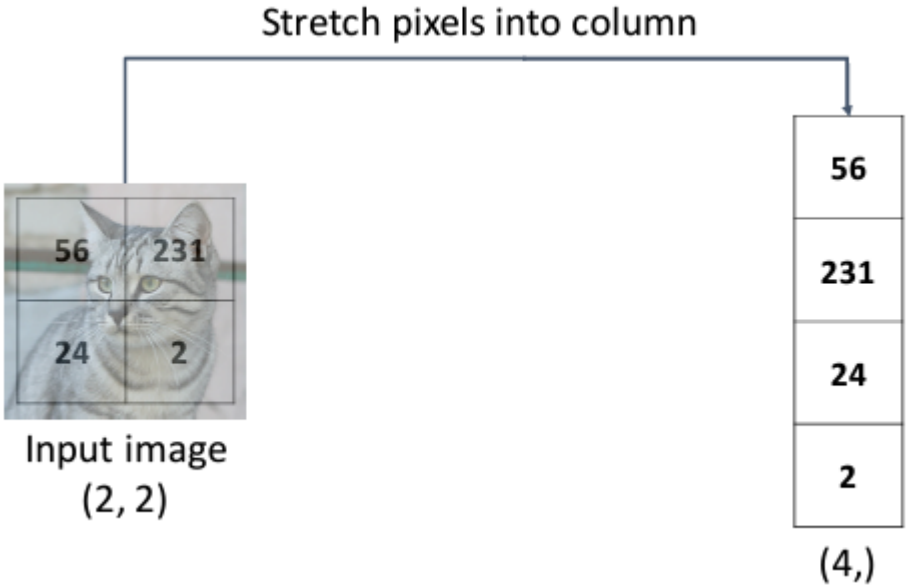
Deep learning basics – image classification with more layers of weights

Fully-connected (linear) layer



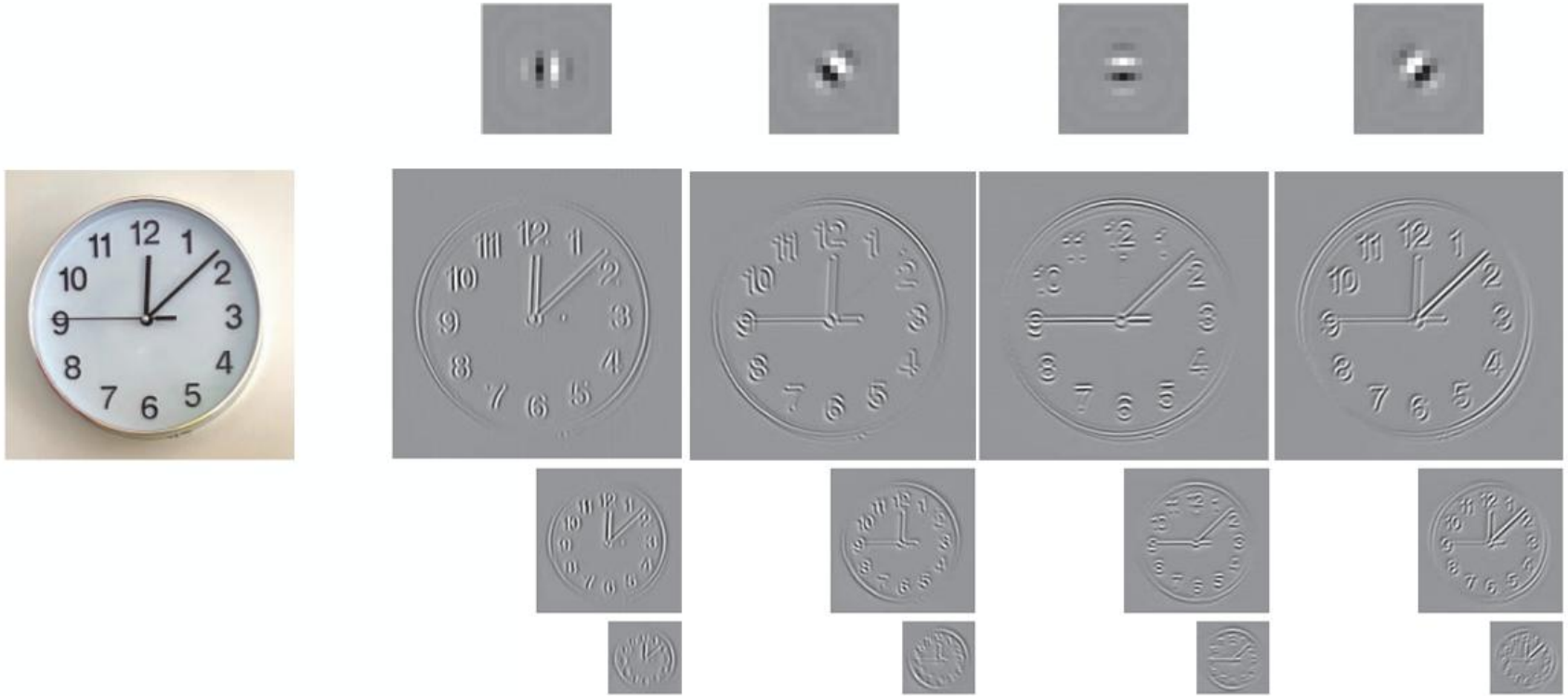
$\mathbf{x} =$ 

- $\mathbf{x}$ is really big! E.g. 256 x 256

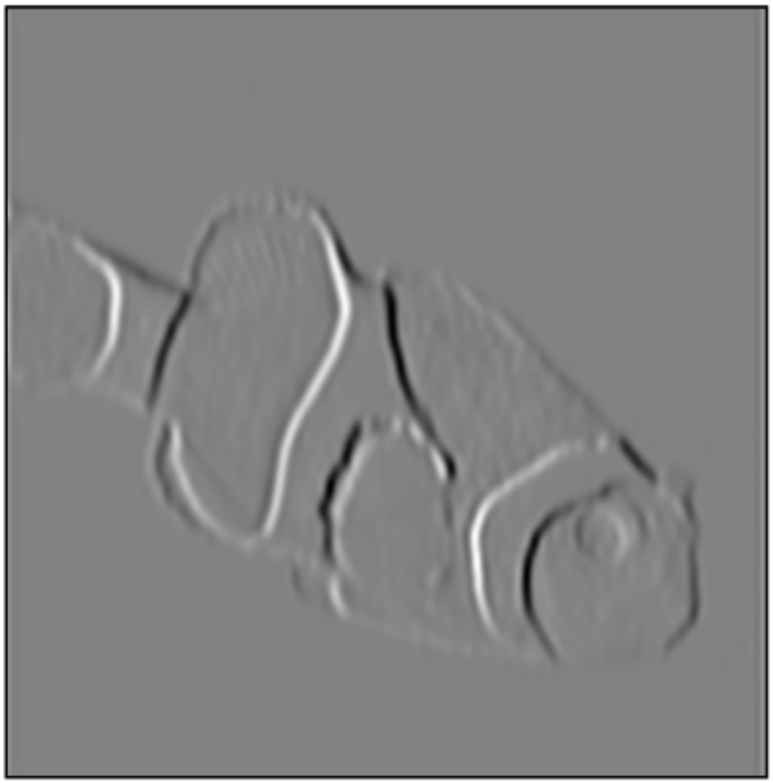


Deep learning basics – convolution kernels

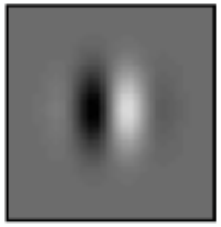
We've already developed linear functions for processing images. They let us do **local** processing at **multiple scales**.



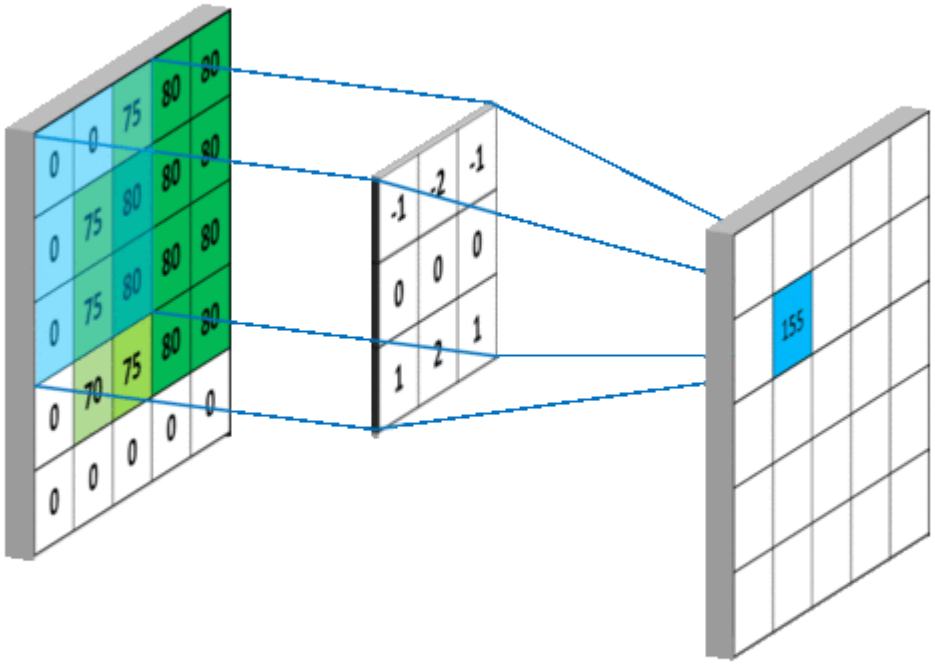
Deep learning basics – convolution kernels



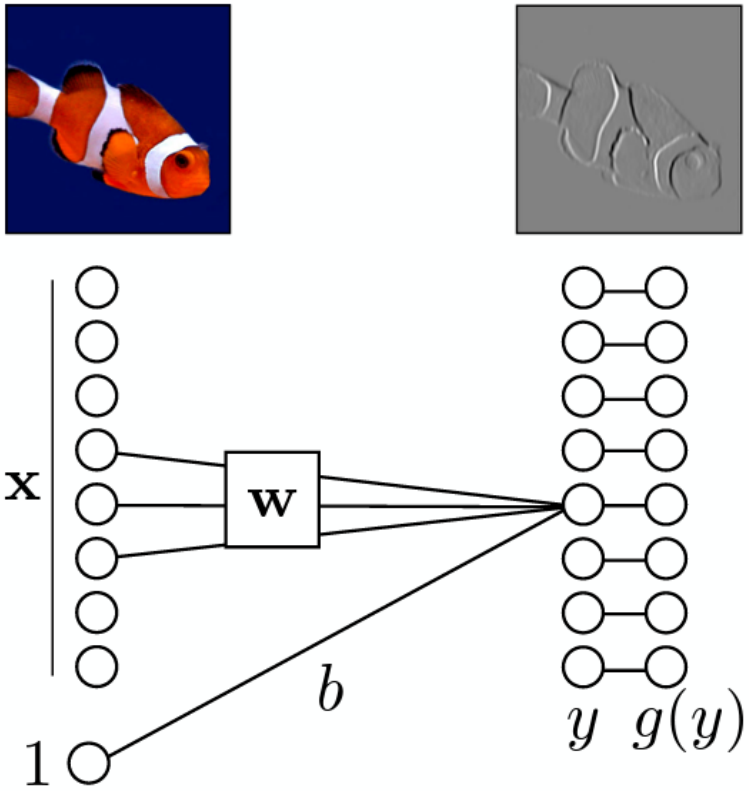
filter



Deep learning basics – convolution kernels

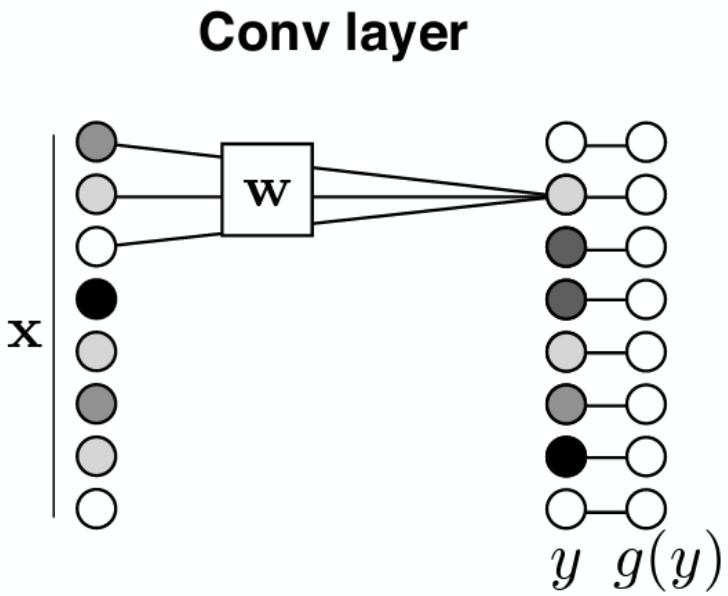


Deep learning basics – convolution kernels as weights



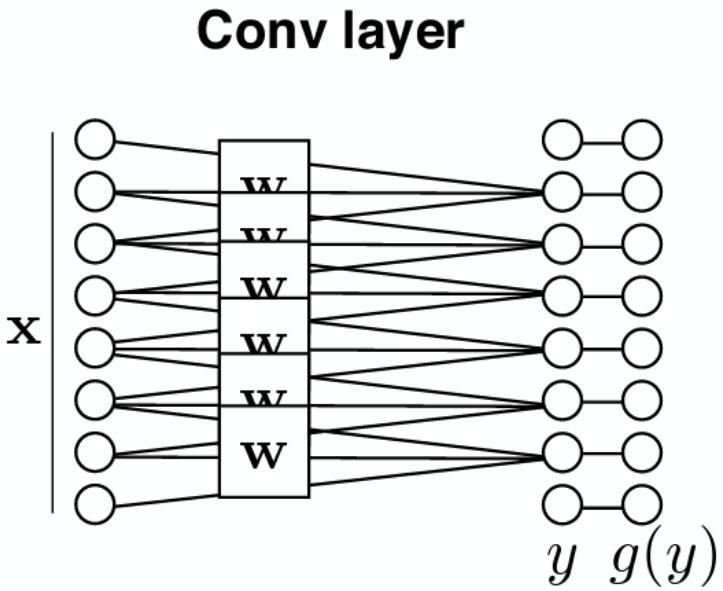
Each unit is connected to a subset of the units in the previous layer.

Deep learning basics – convolution kernels as weights



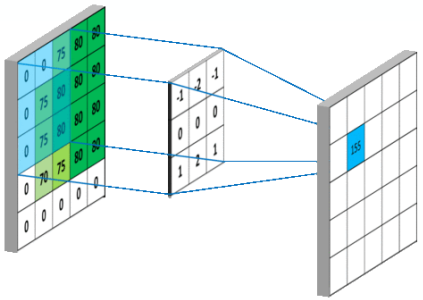
Each output unit is computed from an image patch.

Deep learning basics – convolution kernels shared by all patches



We “share” weights for each patch.

If a feature is useful in one position, it should be useful in others, too.

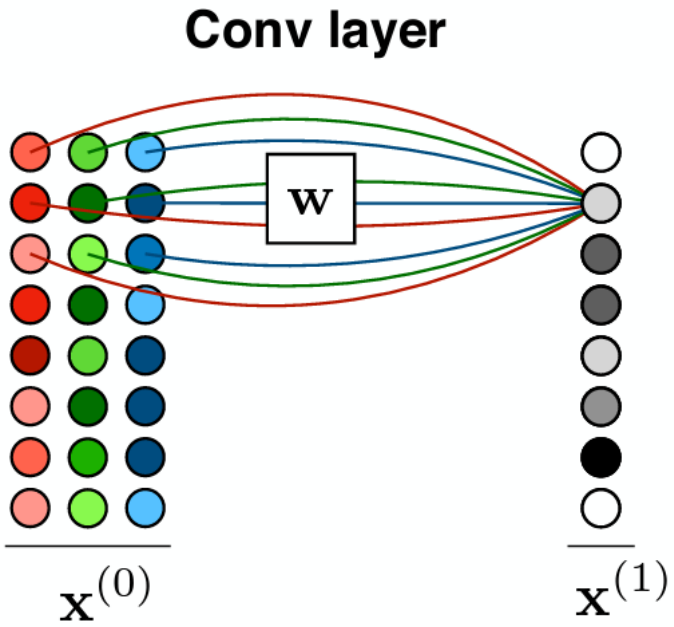


What if we have color?

(And multiple input channels in general)

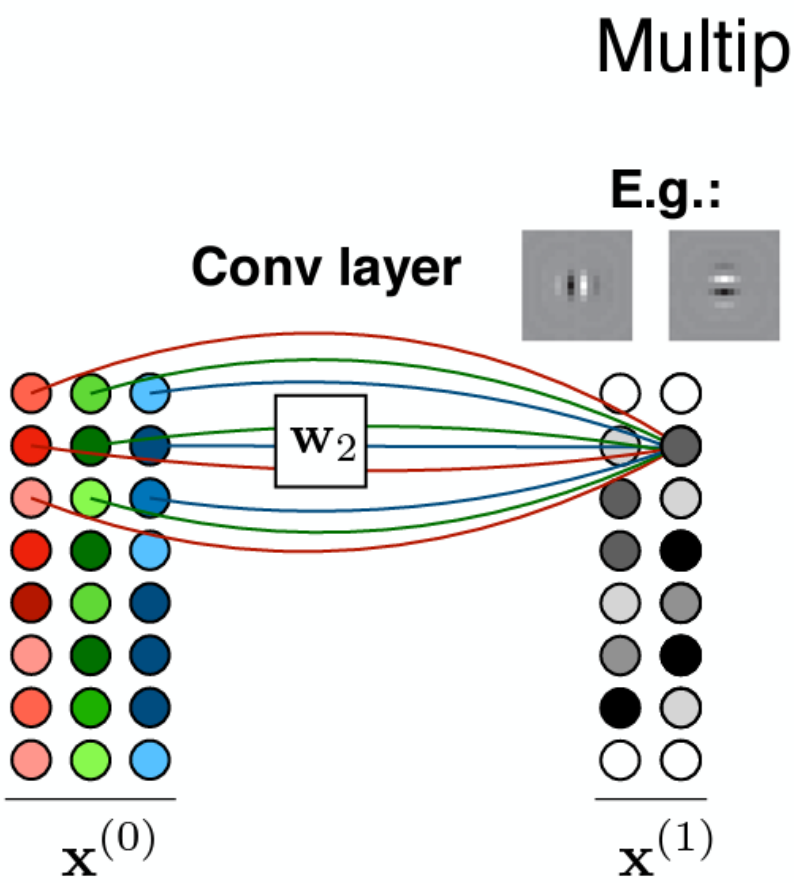
Deep learning basics – convolution on RGB images / multi-channel input

Multiple channels



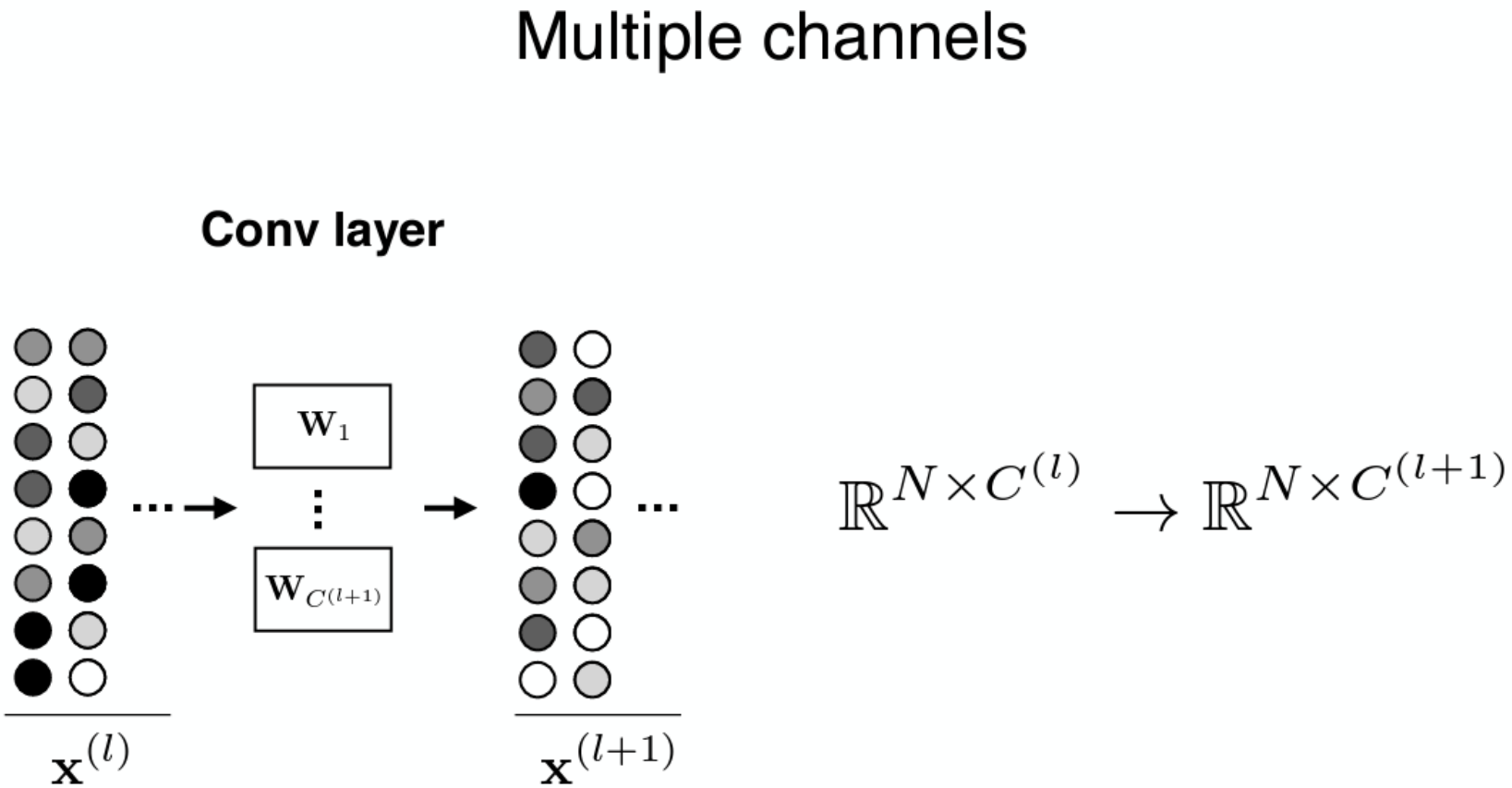
$$\mathbb{R}^{N \times C} \rightarrow \mathbb{R}^{N \times 1}$$

Deep learning basics – convolution on RGB images / multi-channel input

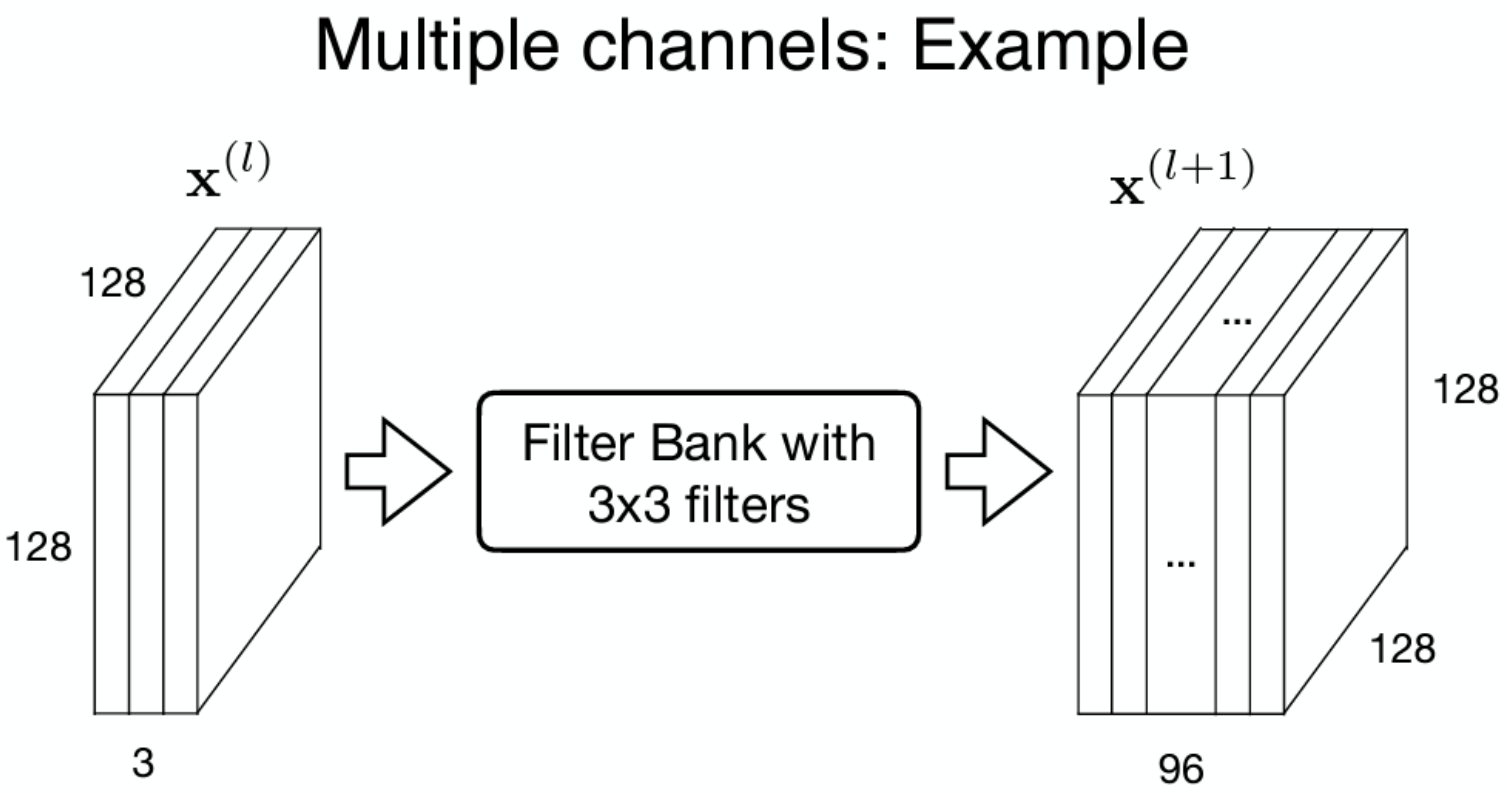


$$\mathbb{R}^{N \times C^{(0)}} \rightarrow \mathbb{R}^{N \times C^{(1)}}$$

Deep learning basics – convolution on RGB images / multi-channel input



Deep learning basics – convolution on RGB images / multi-channel input



How many parameters does *each filter* have?

(a) 9 (b) 27 (c) 96 (d) 2592

Deep learning basics – non-linear transform

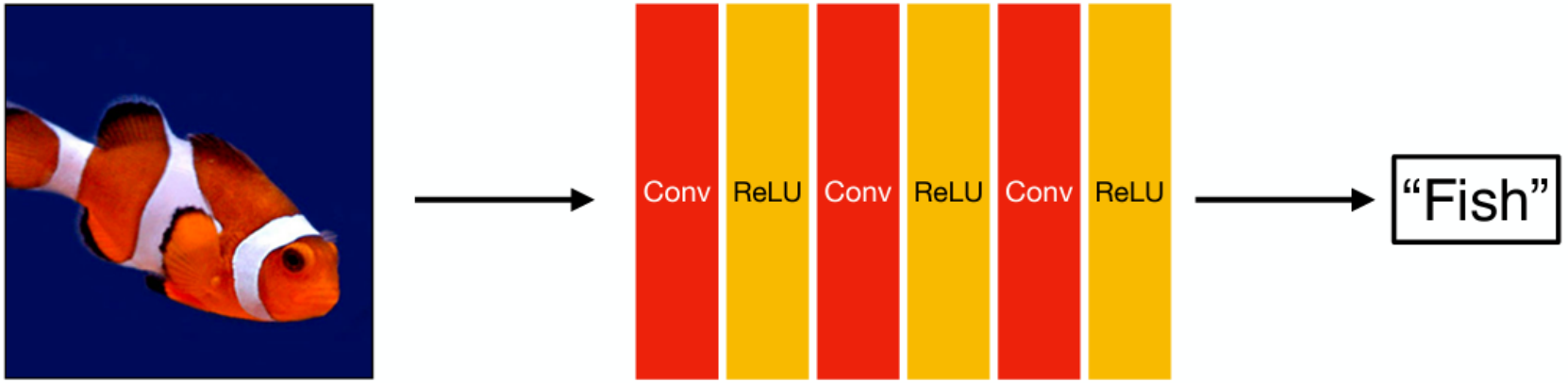
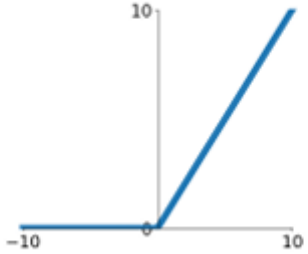


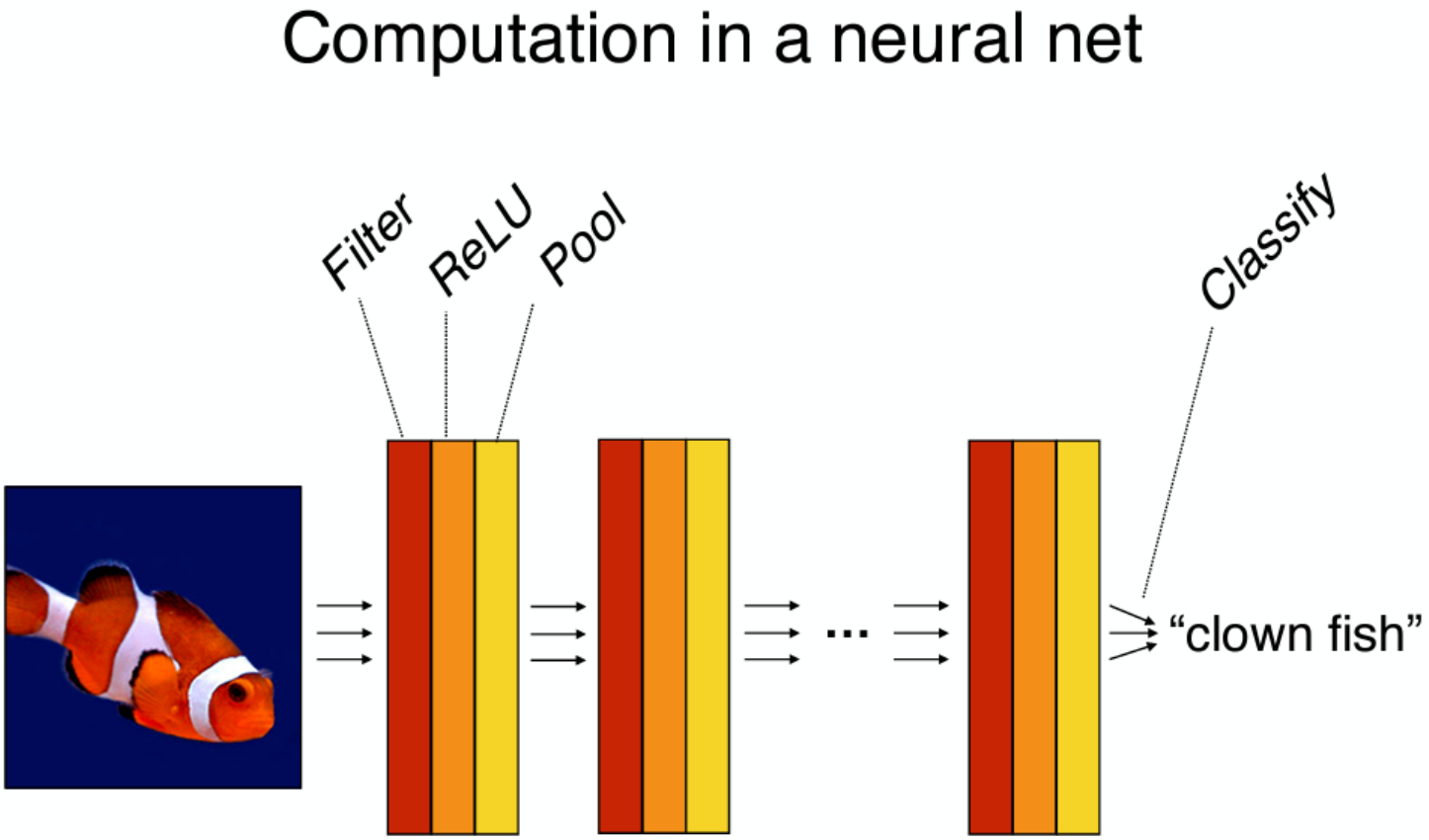
image **x**

Activation Function



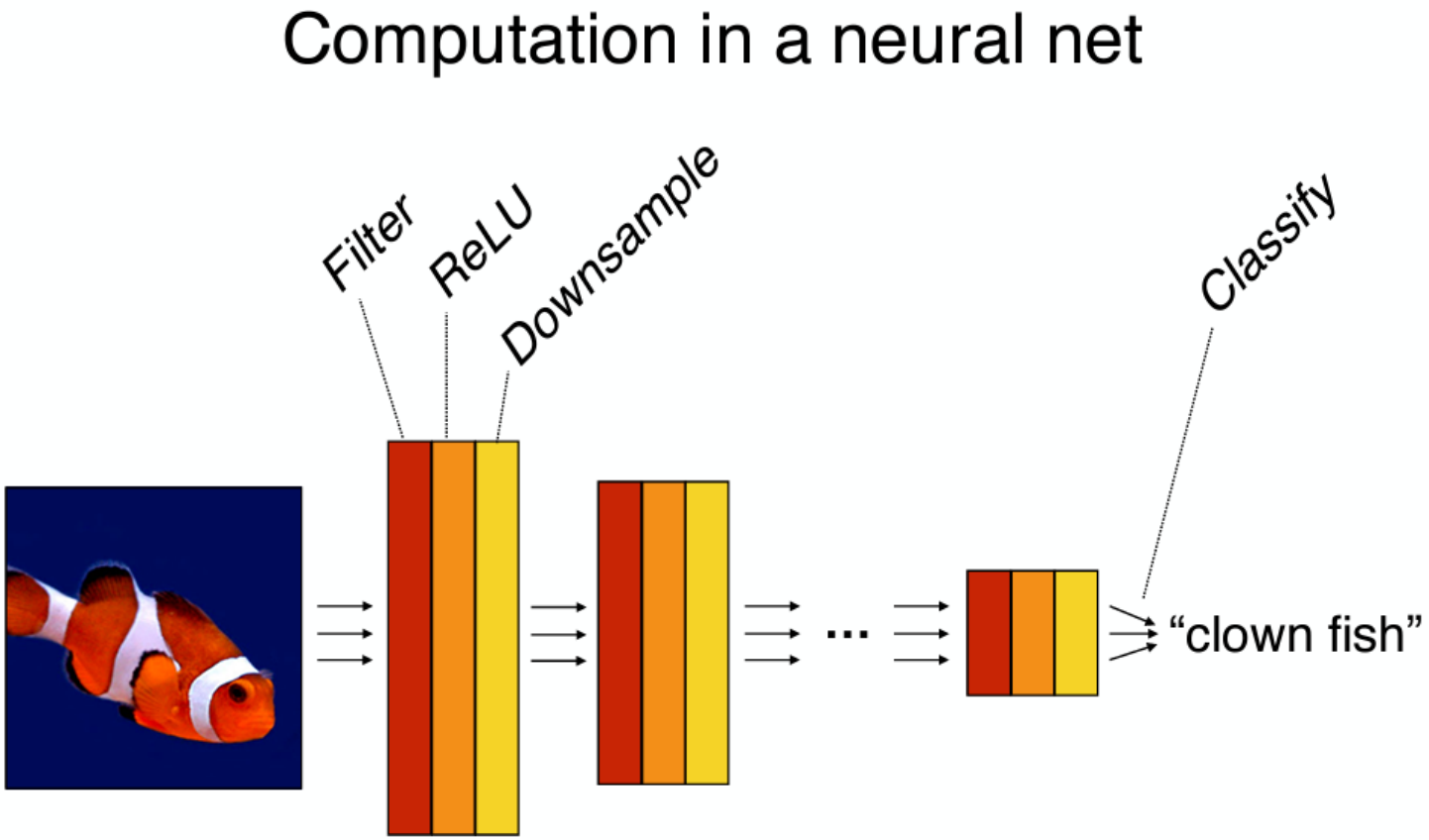
label y

Deep learning basics



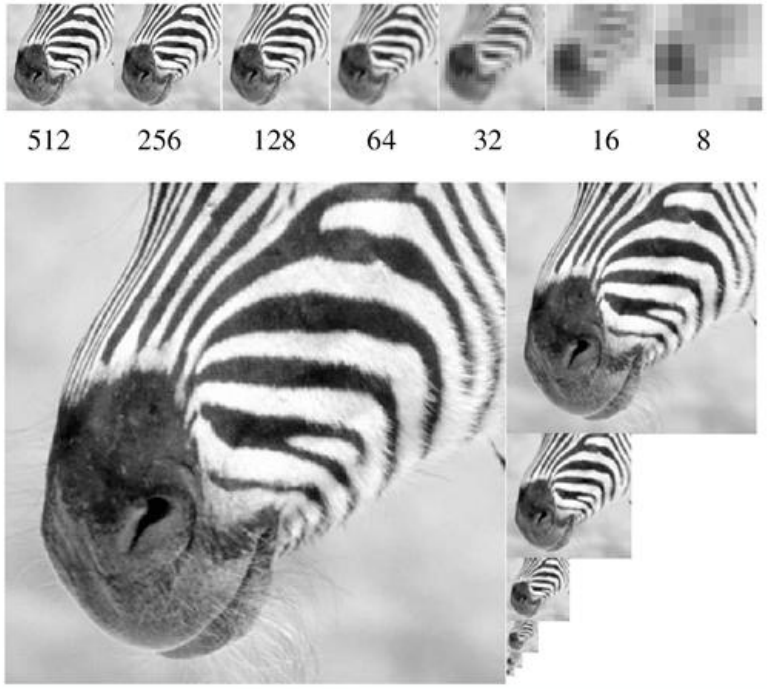
$$f(\mathbf{x}) = f_L(\dots f_2(f_1(\mathbf{x})))$$

Deep learning basics

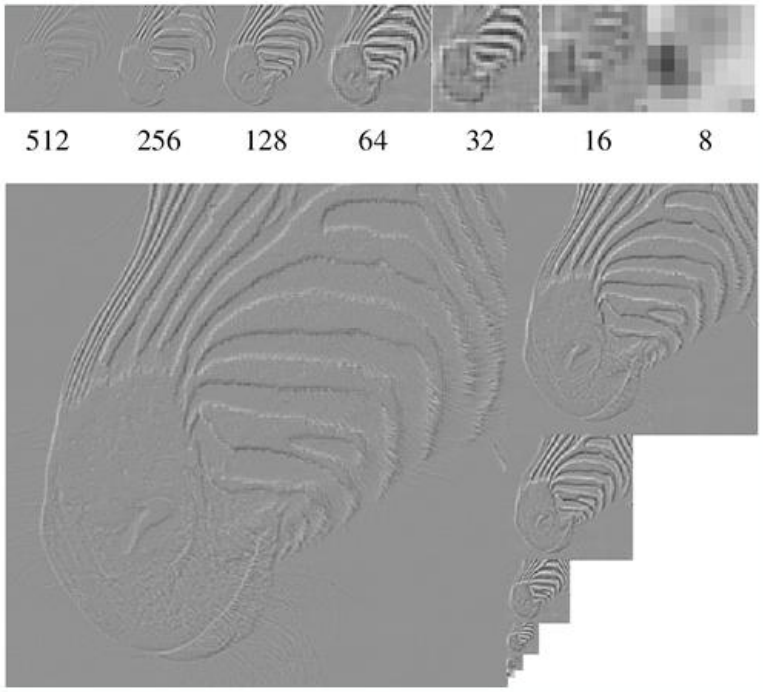


$$f(\mathbf{x}) = f_L(\ldots f_2(f_1(\mathbf{x})))$$

Recall: pyramid representations



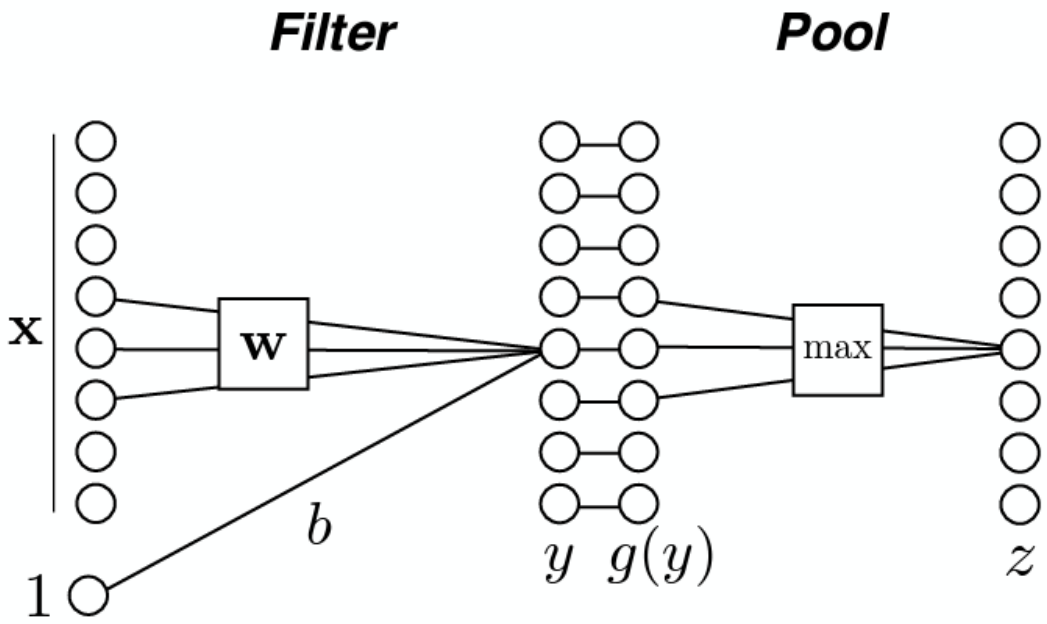
Gaussian Pyramid



Laplacian Pyramid

Can we use a similar idea in CNNs?

Pooling

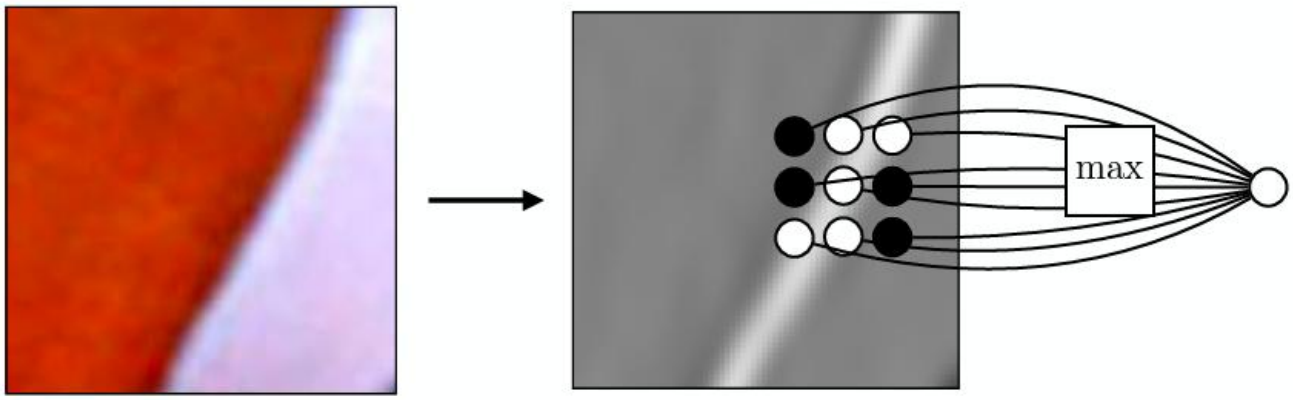


Max pooling

$$z_k = \max_{j \in \mathcal{N}(j)} g(y_j)$$

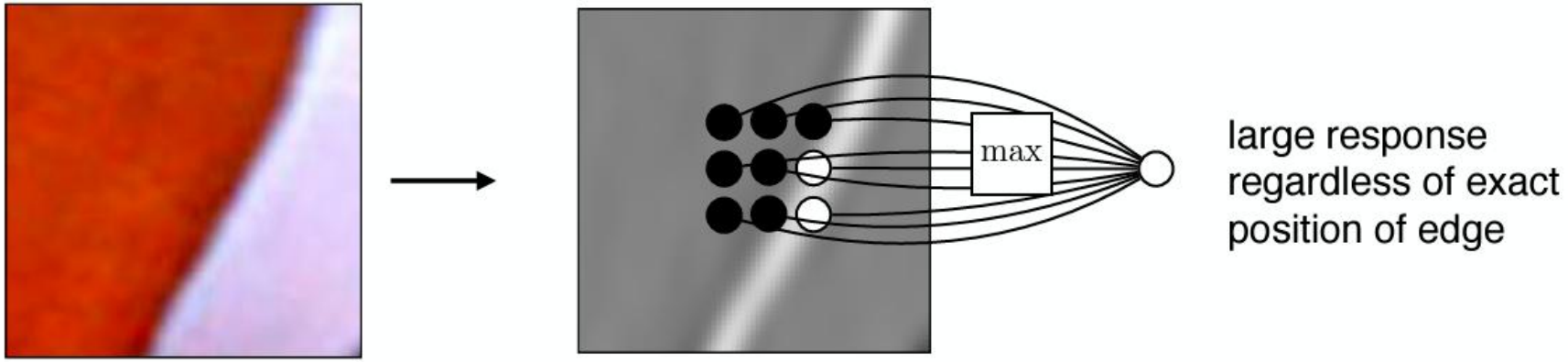
Pooling — Why?

Pooling across spatial locations achieves stability w.r.t. small translations:



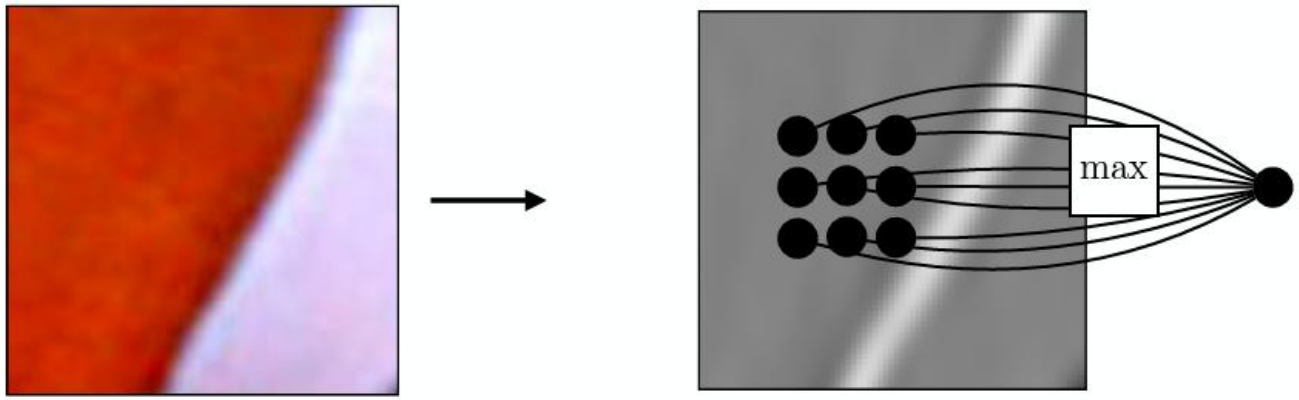
Deep learning basics

Pooling across spatial locations achieves stability w.r.t. small translations:

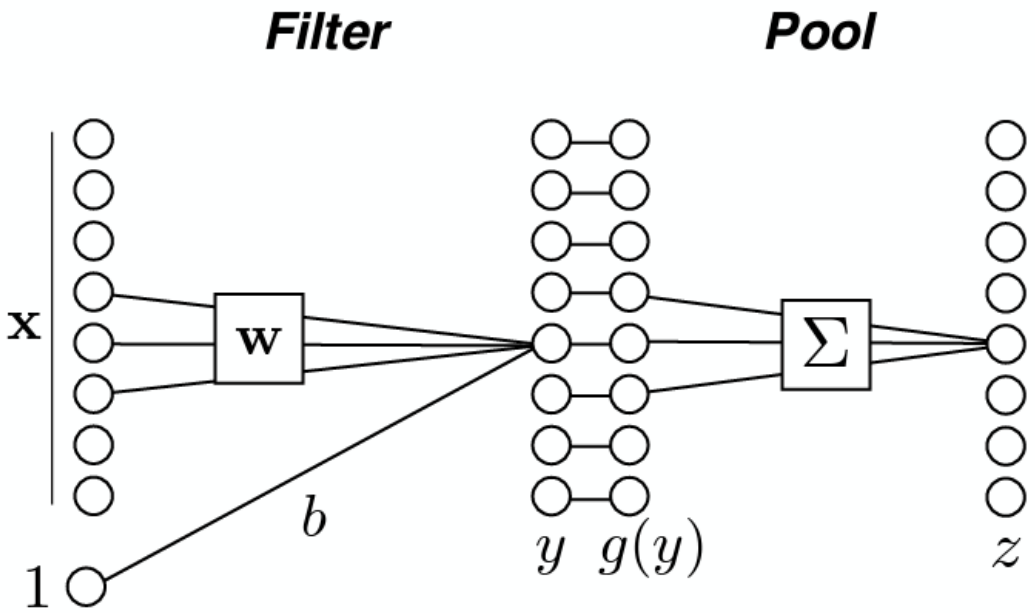


Deep learning basics

Pooling across spatial locations achieves stability w.r.t. small translations:



Pooling



Max pooling

$$z_k = \max_{j \in \mathcal{N}(j)} g(y_j)$$

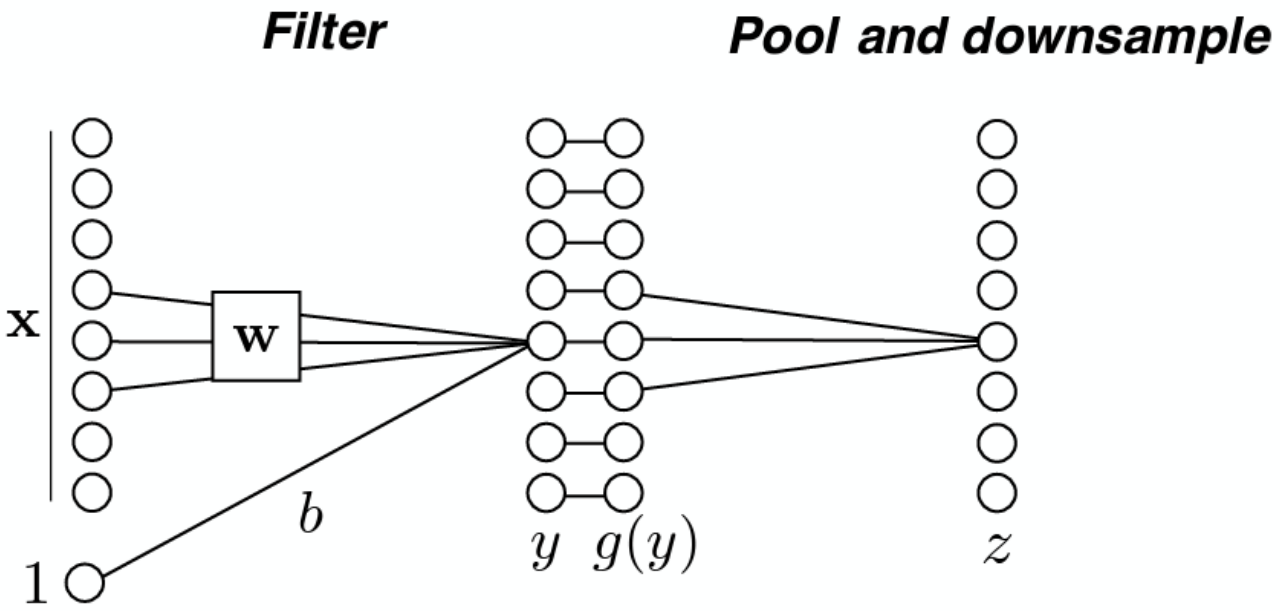
Mean pooling

$$z_k = \frac{1}{|\mathcal{N}|} \sum_{j \in \mathcal{N}(j)} g(y_j)$$

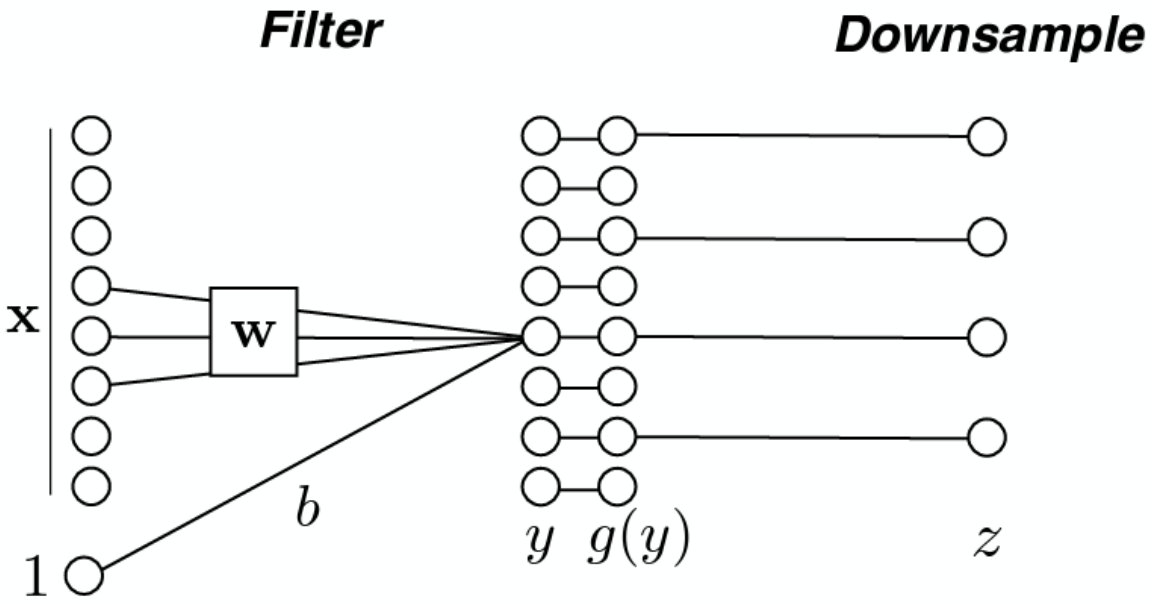
Blurring [Zhang 2019]

$$z = \mathrm{conv}(y, \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix})$$

Downsampling

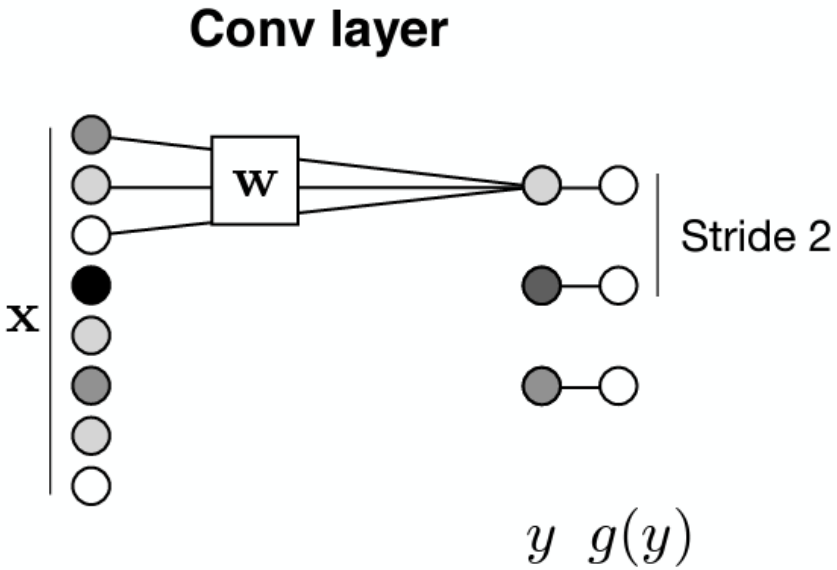


Downsampling



$$\mathbb{R}^{H^{(l)} \times W^{(l)} \times C^{(l)}} \rightarrow \mathbb{R}^{H^{(l+1)} \times W^{(l+1)} \times C^{(l+1)}}$$

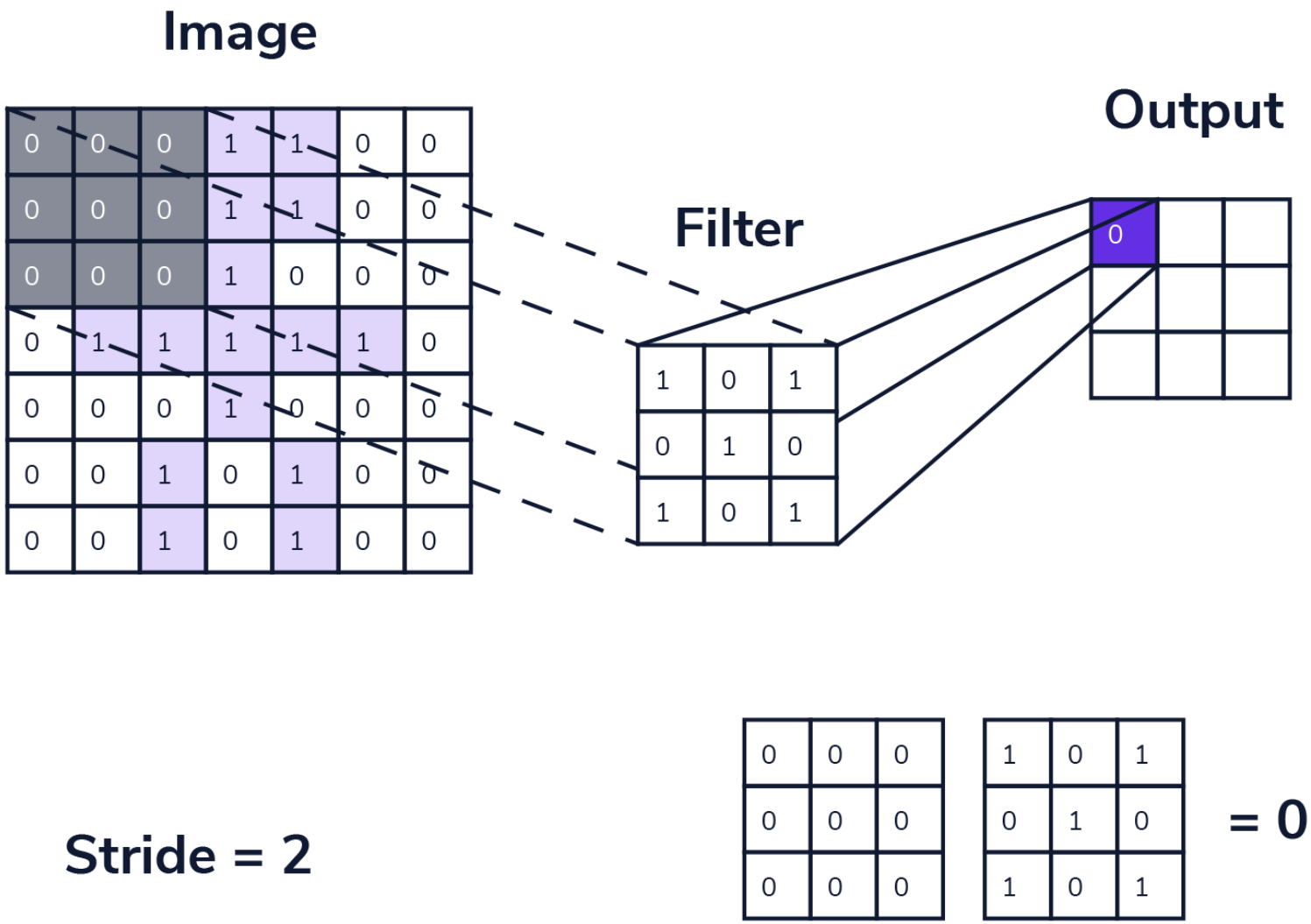
Strided operations



Strided operations combine a given operation (convolution or pooling) and downsampling into a single operation.

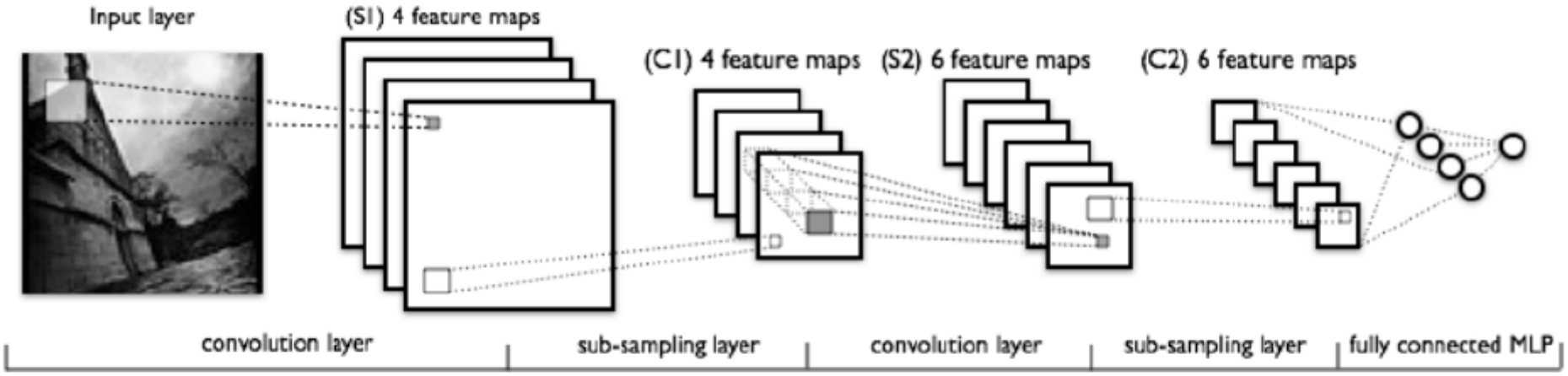
Strided convolution is an alternative to pooling layers: just do a strided convolution!

Deep learning basics – convolution kernels



Network designs

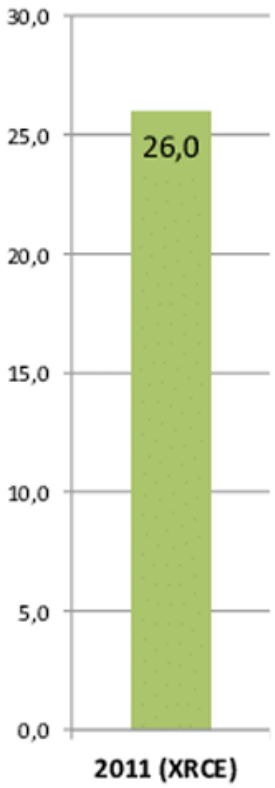
Convolutional neural nets



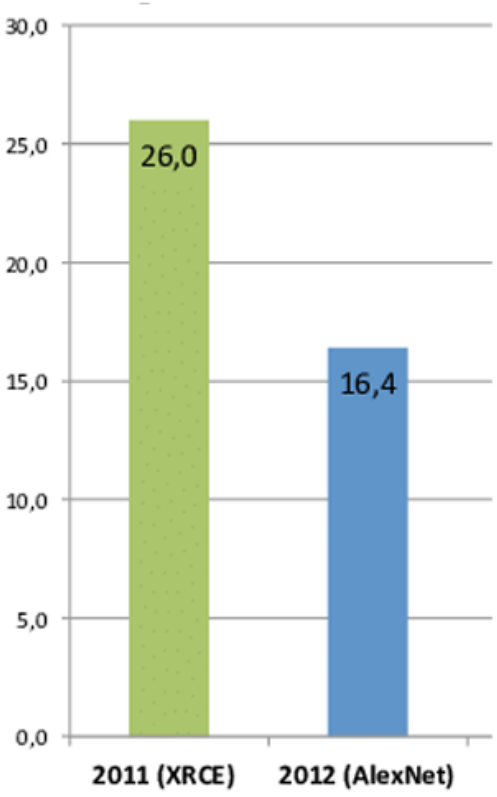
$$G = \max(0, F)$$

Elementwise rectification or "RELU" - rectified linear unit

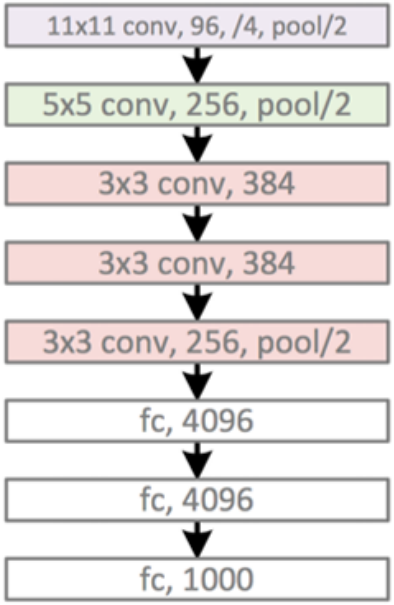
ImageNet classification (top 5)



ImageNet classification (top 5)



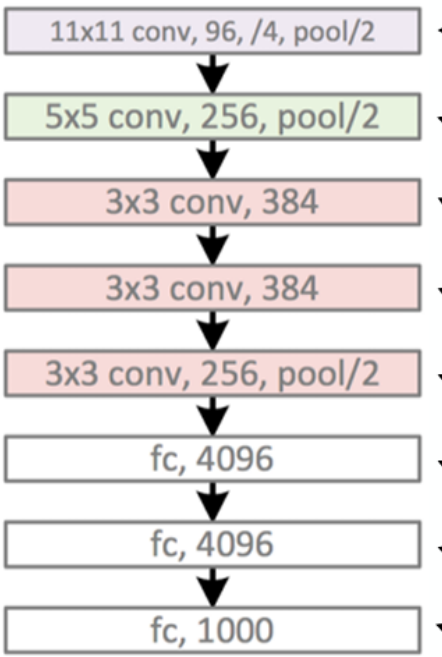
2012: AlexNet
5 conv. layers



Error: 16.4%

[Krizhevsky et al: ImageNet Classification with Deep Convolutional Neural Networks, NeurIPS 2012]

Alexnet — [Krizhevsky et al. NIPS 2012]



[227x227x3] INPUT

[55x55x96] CONV1: 96 11x11 filters at stride 4, pad 0
[27x27x96] MAX POOL1: 3x3 filters at stride 2
[27x27x96] NORM1: Normalization layer

[27x27x256] CONV2: 256 5x5 filters at stride 1, pad 2
[13x13x256] MAX POOL2: 3x3 filters at stride 2
[13x13x256] NORM2: Normalization layer

[13x13x384] CONV3: 384 3x3 filters at stride 1, pad 1

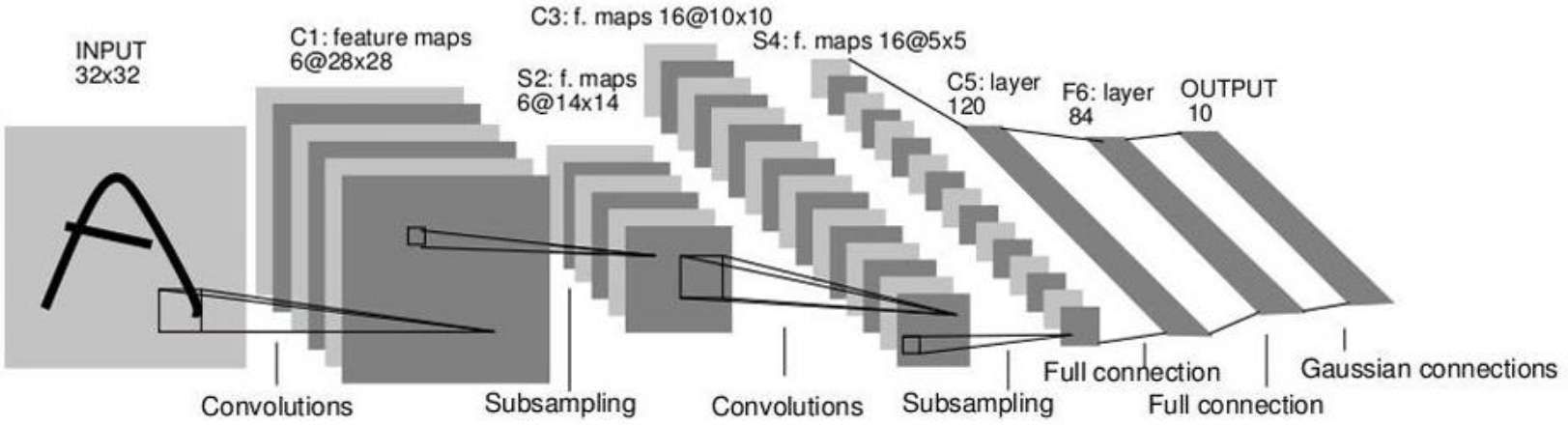
[13x13x384] CONV4: 384 3x3 filters at stride 1, pad 1

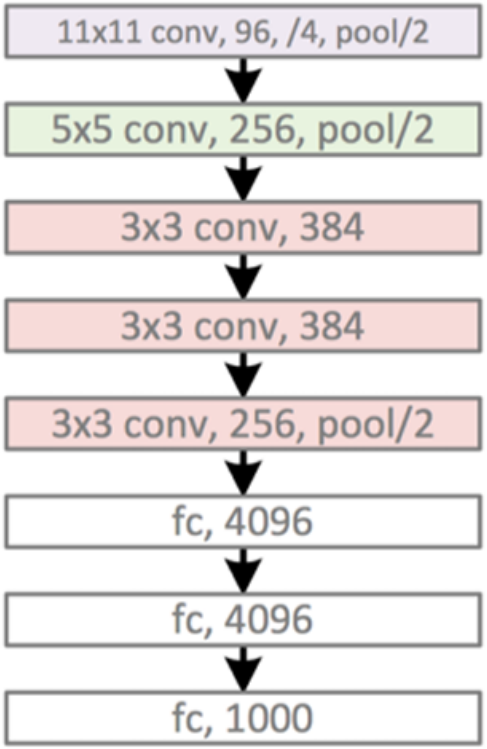
[13x13x256] CONV5: 256 3x3 filters at stride 1, pad 1
[6x6x256] MAX POOL3: 3x3 filters at stride 2

[4096] FC6: 4096 neurons

[4096] FC7: 4096 neurons

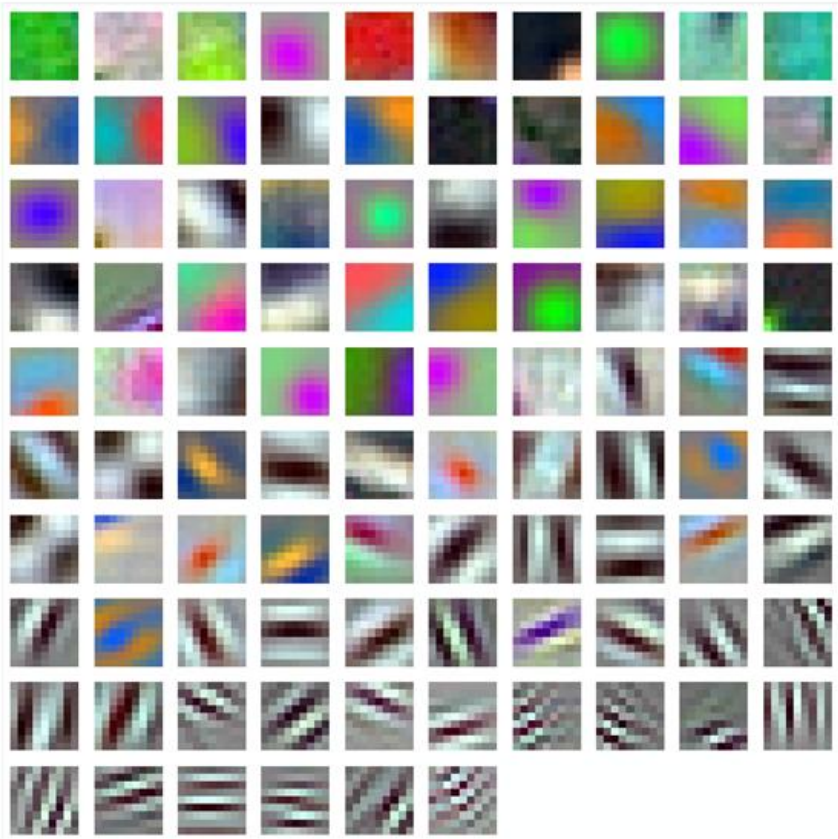
[1000] FC8: 1000 neurons (class scores)





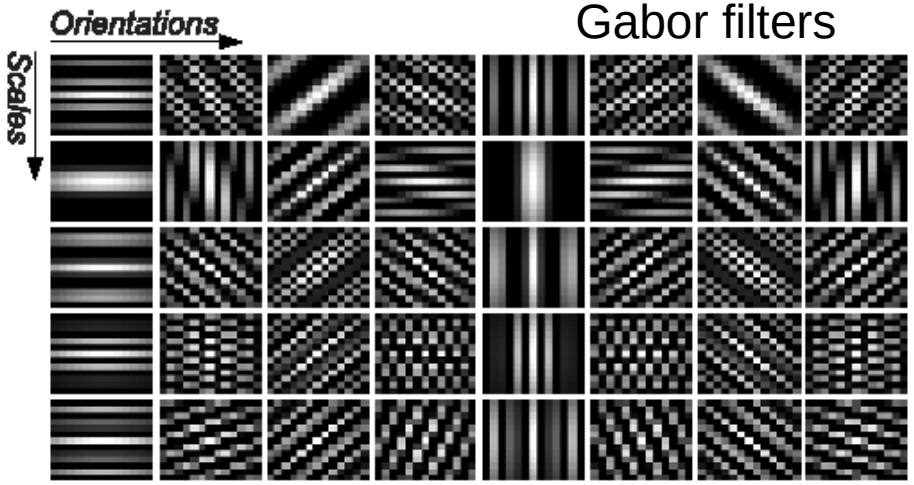
What filters are learned?

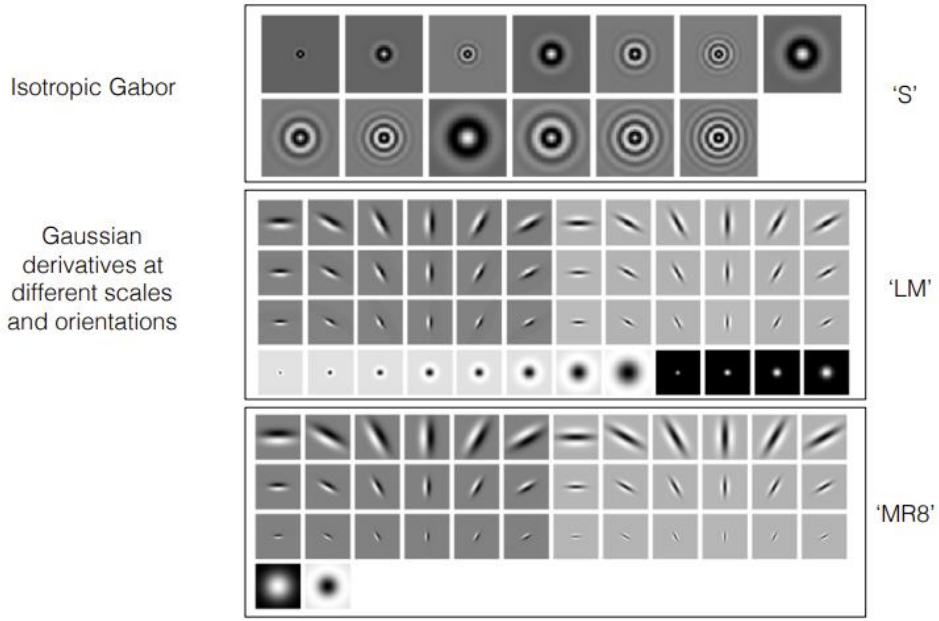
Filters in first layer



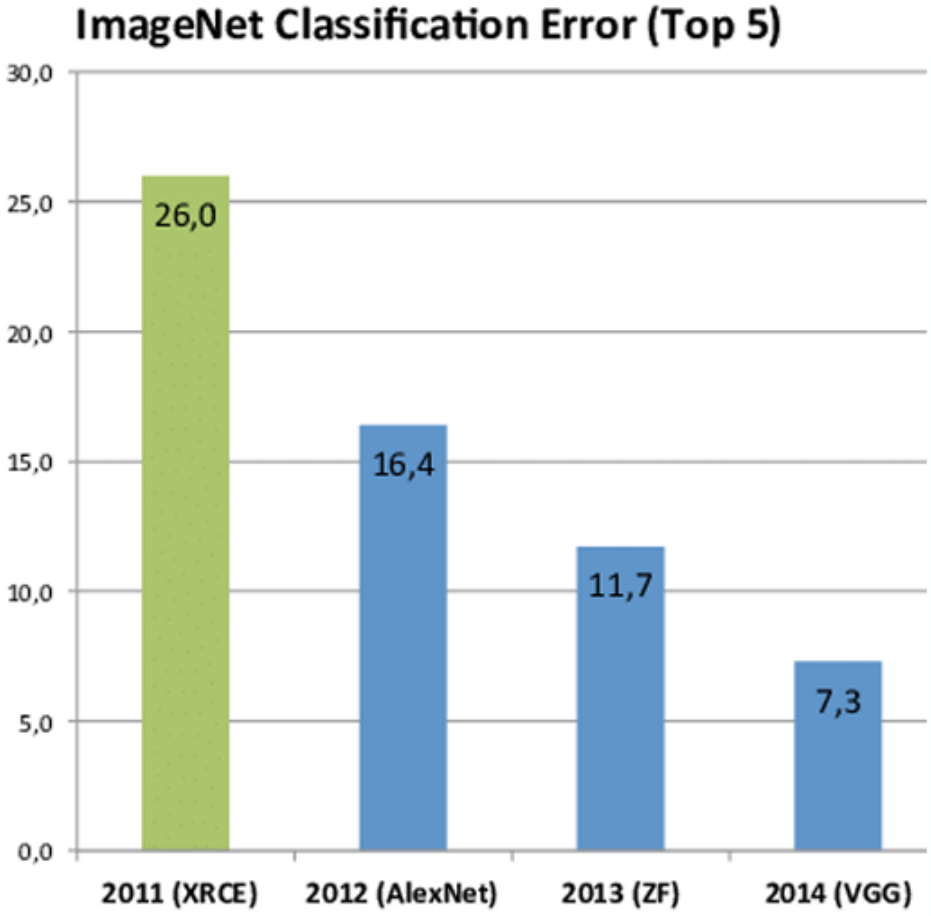
96 Units in conv1

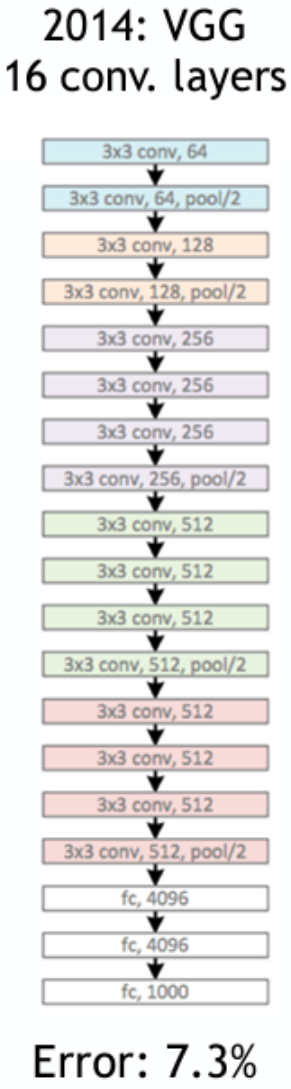
Gabor filters





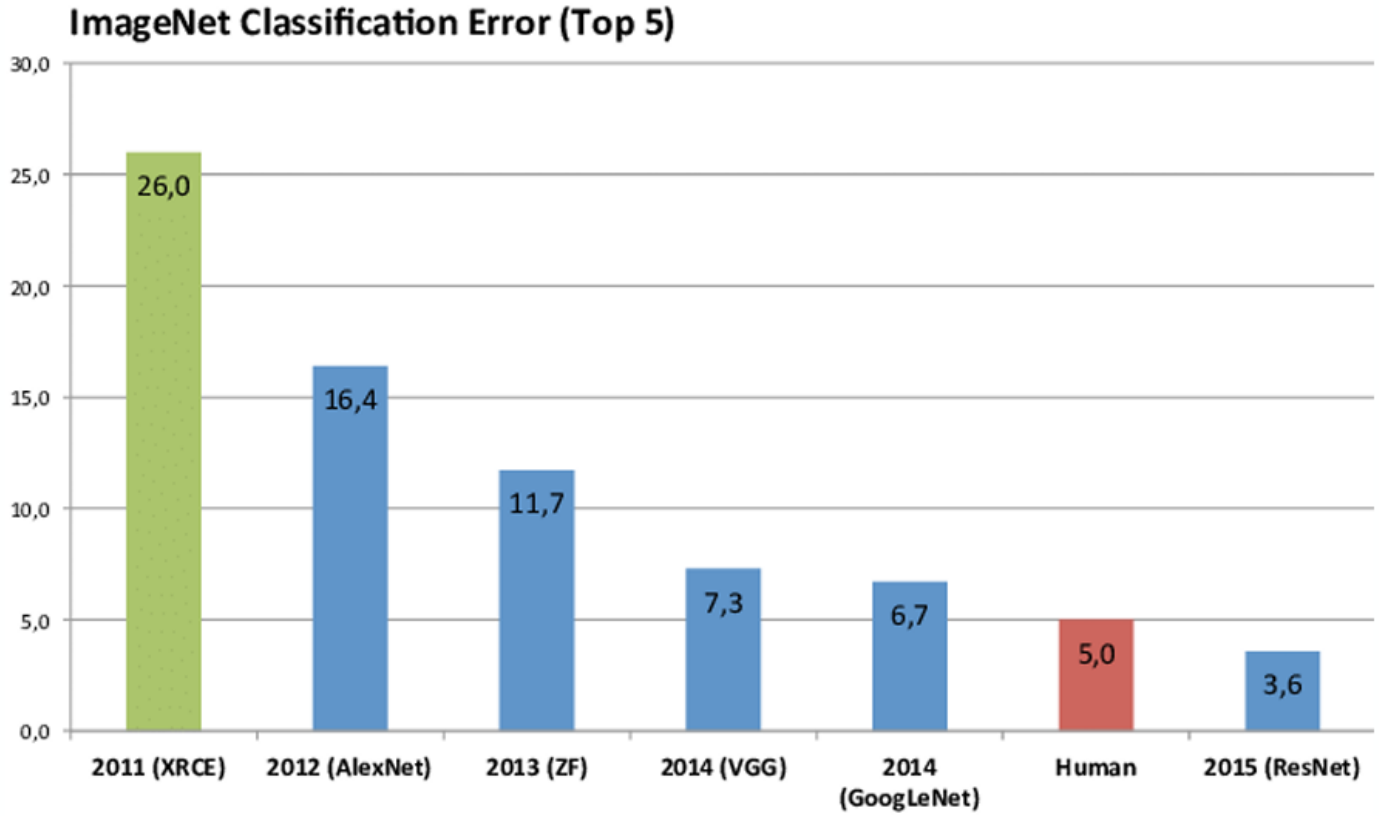
Very deep CNN





Error: 7.3%

Very deep CNN



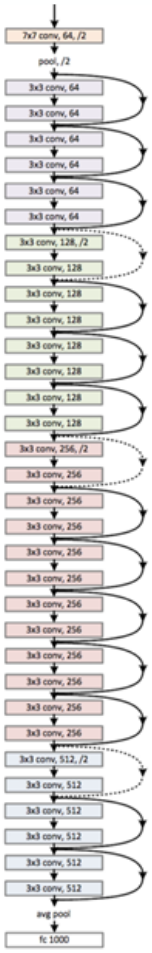
2016: ResNet
>100 conv. layers

Error: 3.6%

Very deep CNN

2016: ResNet
>100 conv. layers

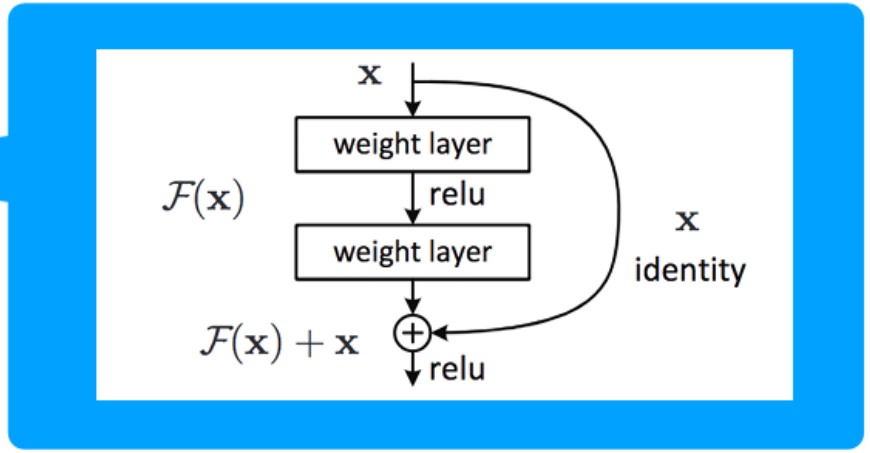
Error: 3.6%



ResNet [He et al, 2016]

Main developments

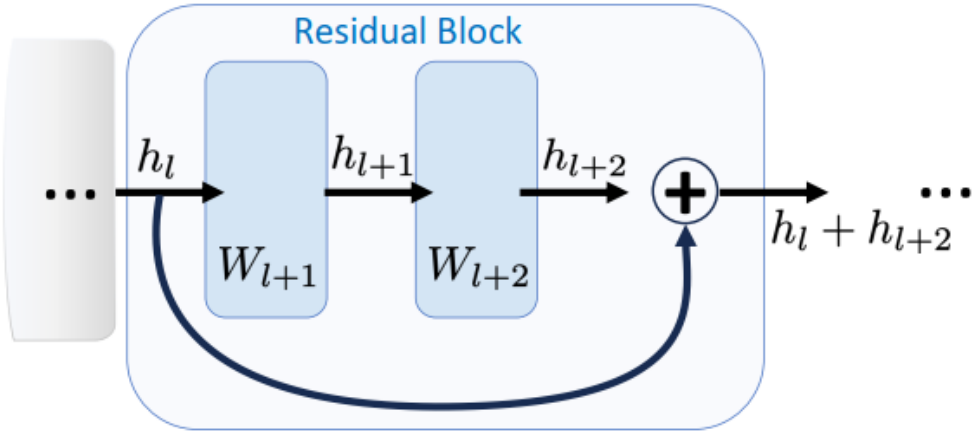
- Increased depth possible through residual blocks



Residual Networks

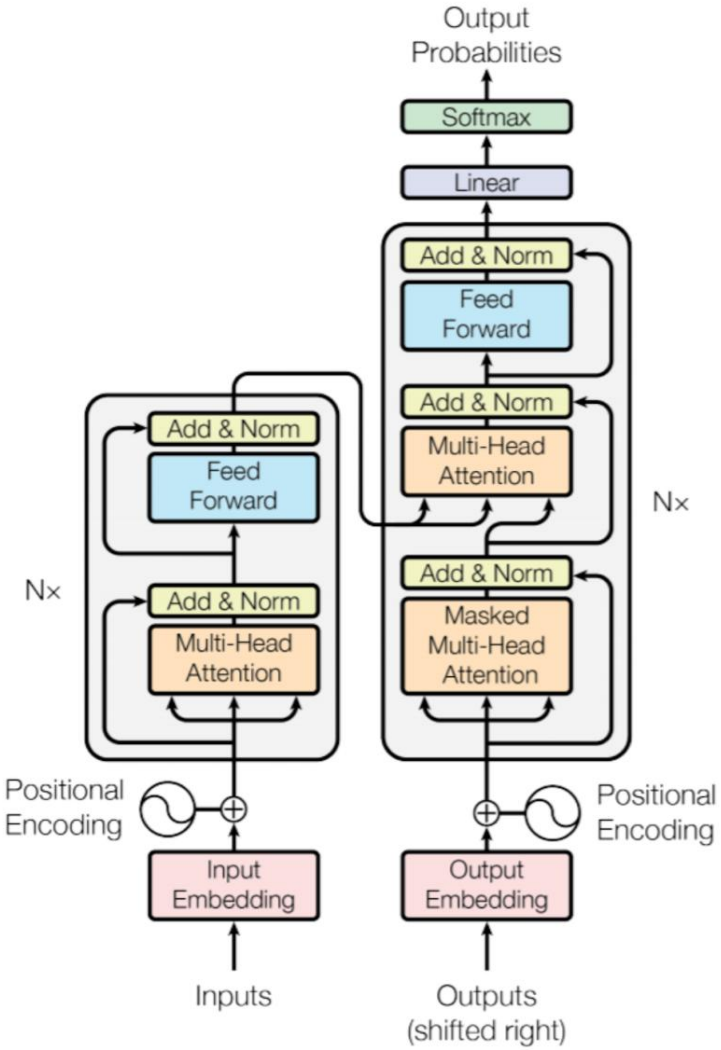
- In practice, deep networks can work worse than shallow ones!
  - Simple / shallow transforms are easier to find and may already work well
  - Transforming more not helpful – have to "find" identity transform?

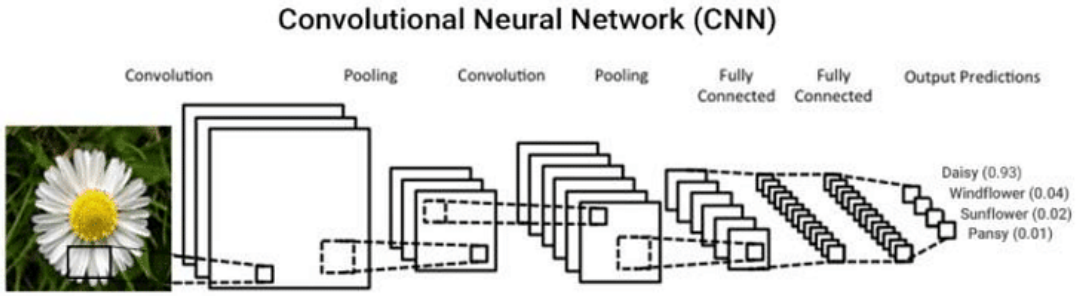
- Residual Block
  - Use layers to estimate "update" to features, rather than transform
  - If the block's weights W are near zero, just keeps input representation
  - Helps with "vanishing gradient" problem: h_l has direct effect on output



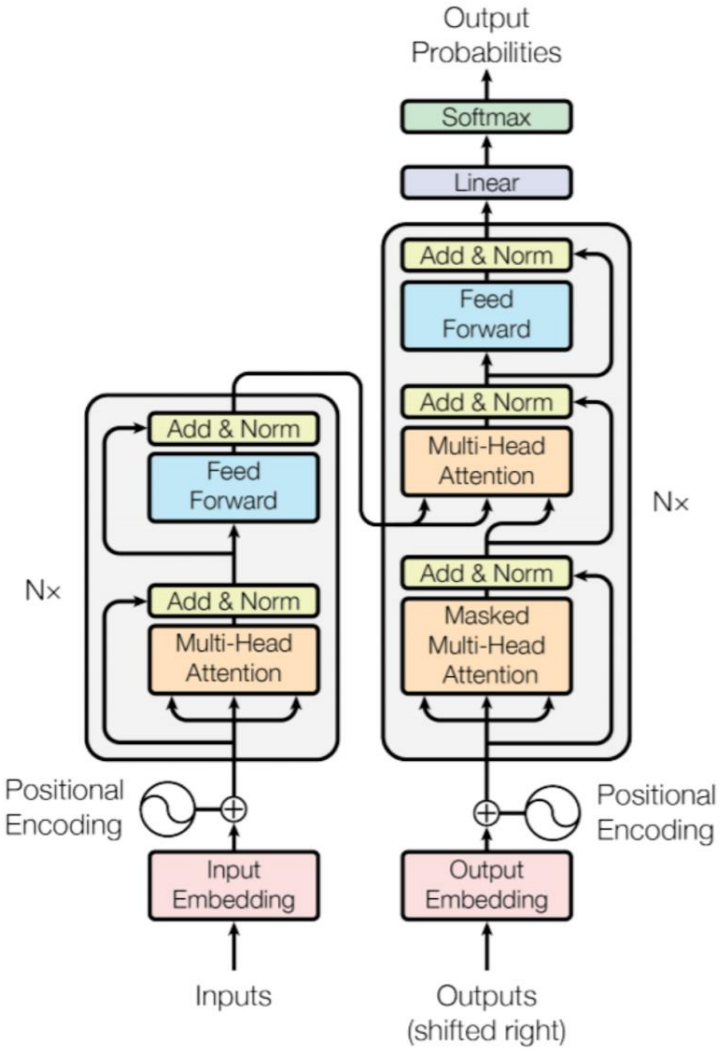
Residual blocks:
- allow "deep" networks
- early layers train better
- unhelpful layers can learn to "skip" (zero value)
- Can use with any layer type (convolutional, dense, etc.)

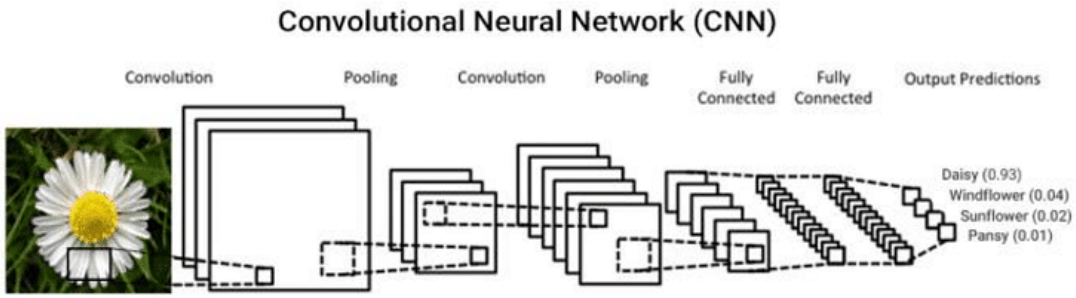
Transformer vs. CNN





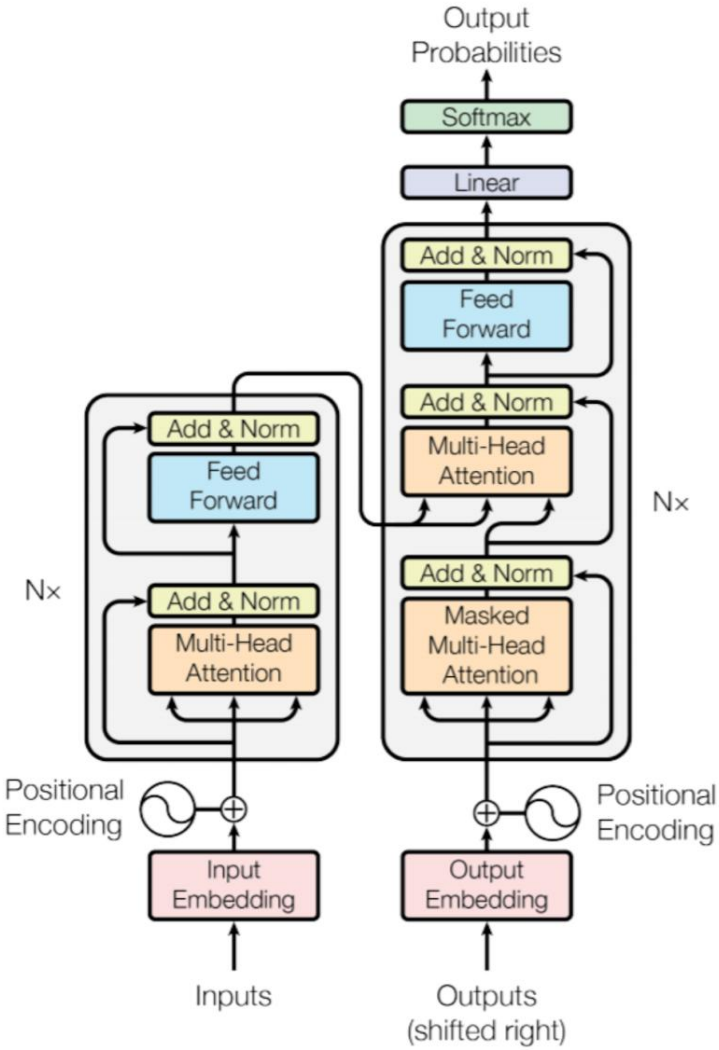
Transformer vs. CNN

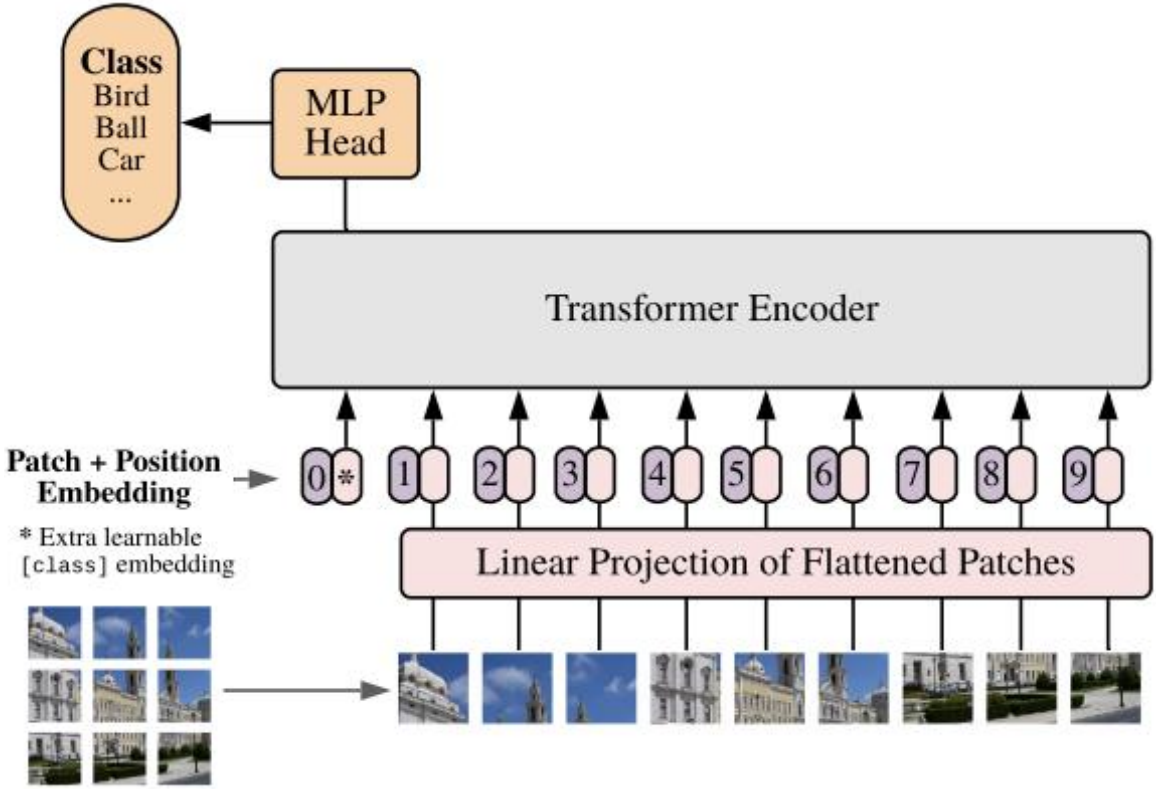




GPT = Generative Pre-trained Transformer

Vision Transformer (ViT)

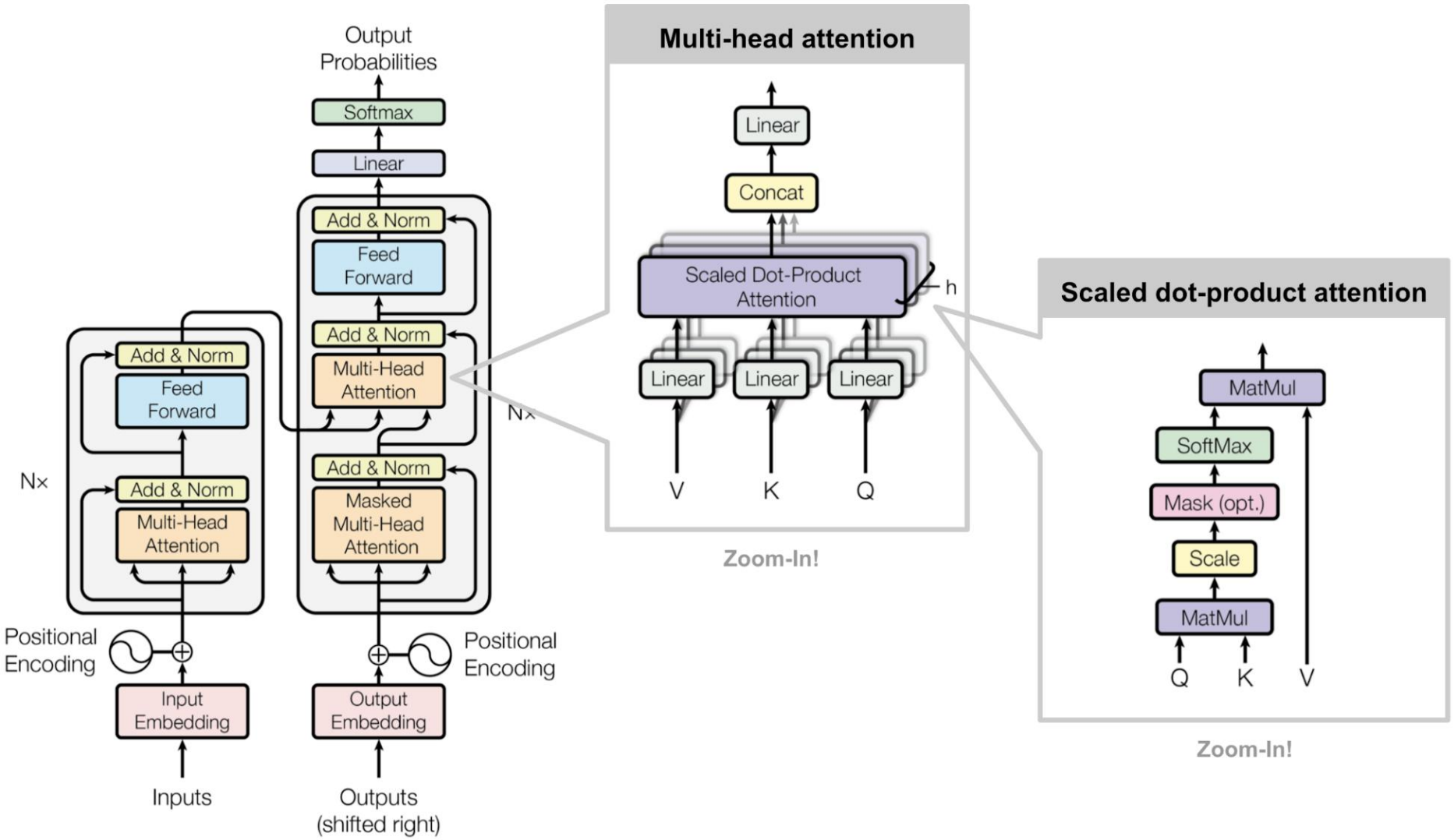




GPT = Generative Pre-trained Transformer

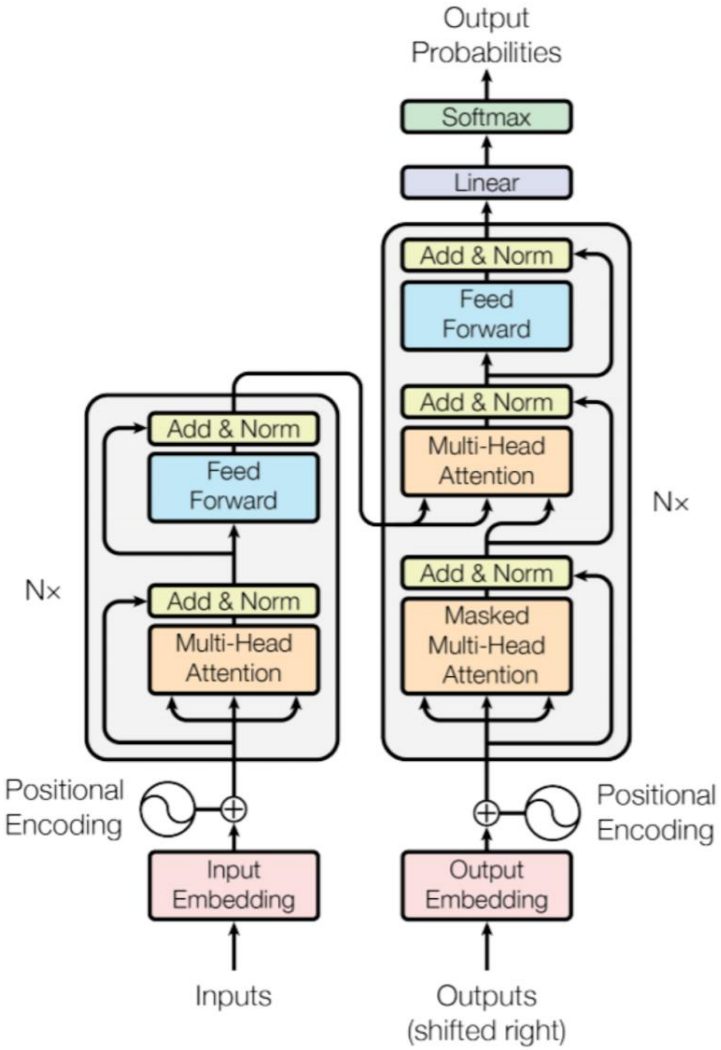
Vision Transformer (ViT)

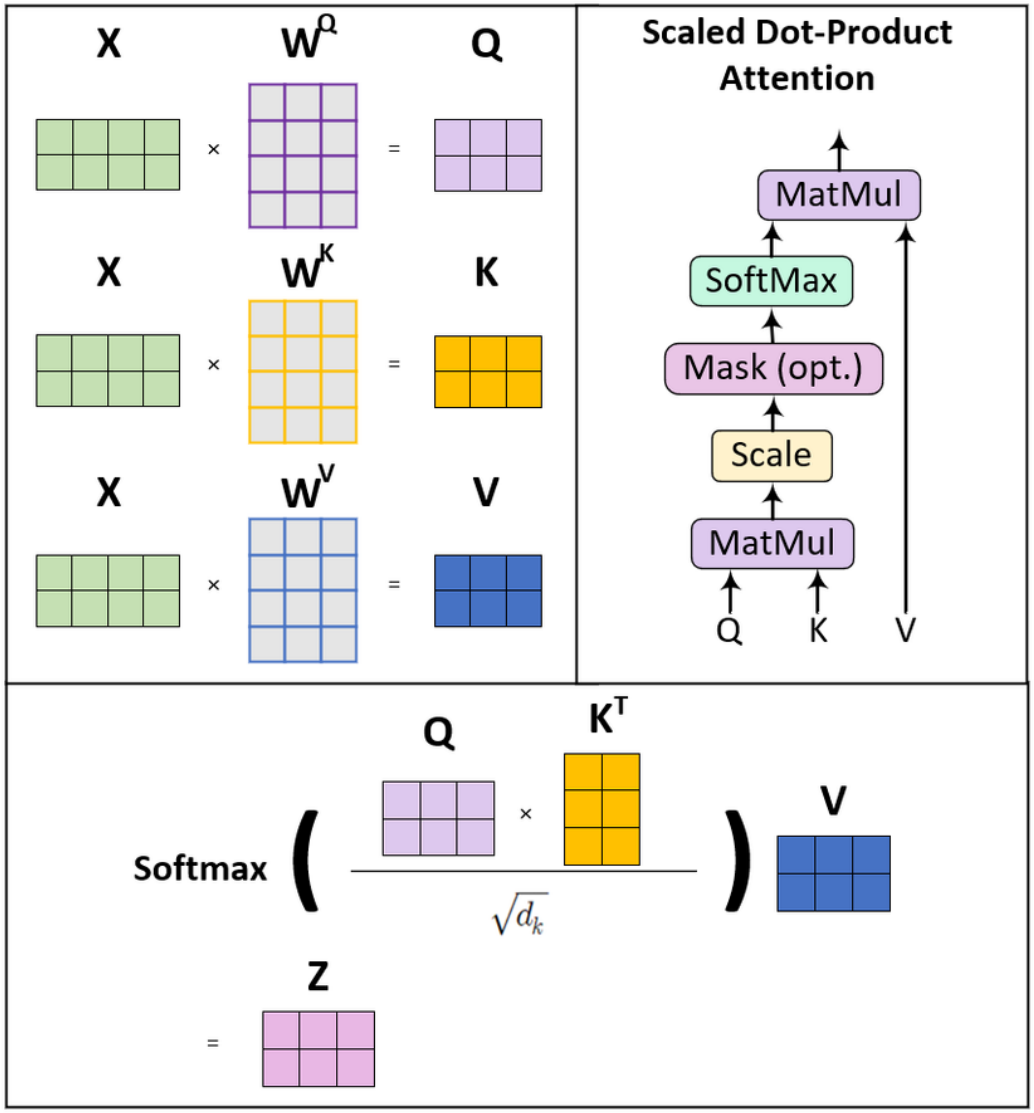
$$\text{attention} = \text{softmax}(\frac{\mathbf{QK}^T}{\sqrt{d_k}})\mathbf{V}$$



Vision Transformer (ViT)

$$\text{attention} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V}$$





Attention Block

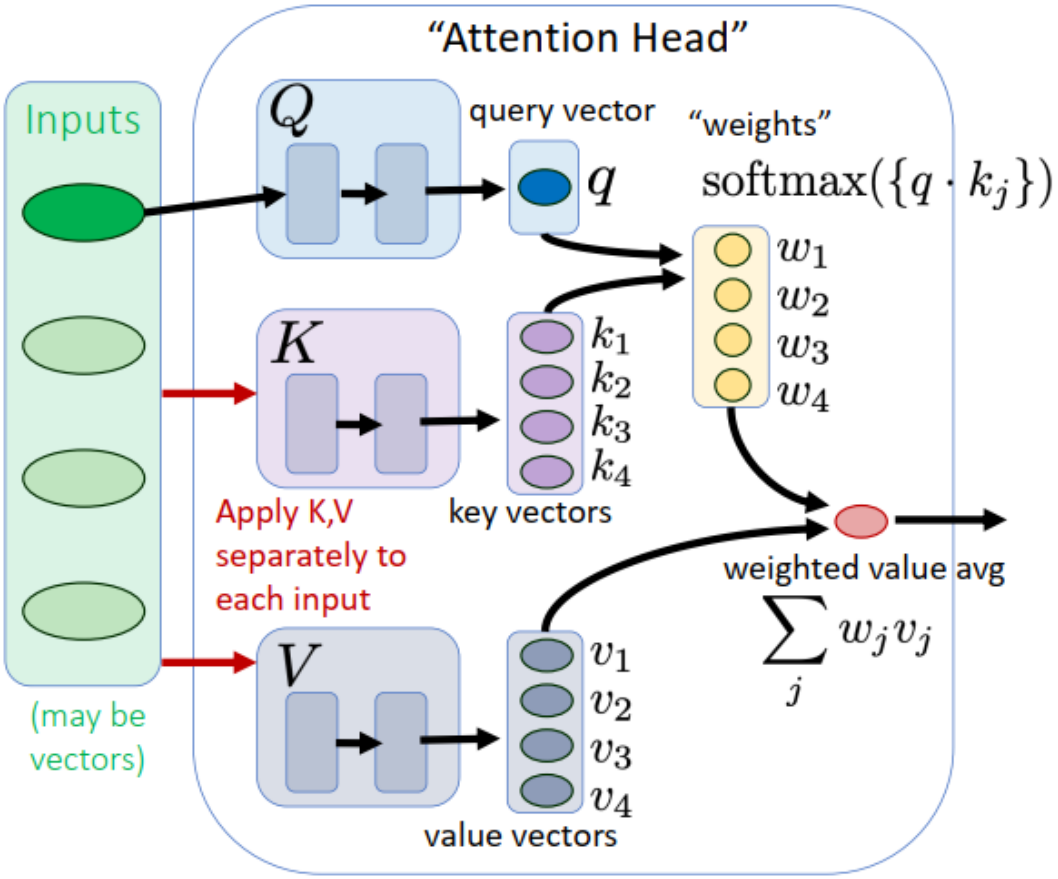
- Extract representation from many (mostly irrelevant?) inputs
  - Ex: predict next work from past sequence of observed words

Inputs: a collection of (possibly vector-valued) measurements

Query: context for current prediction

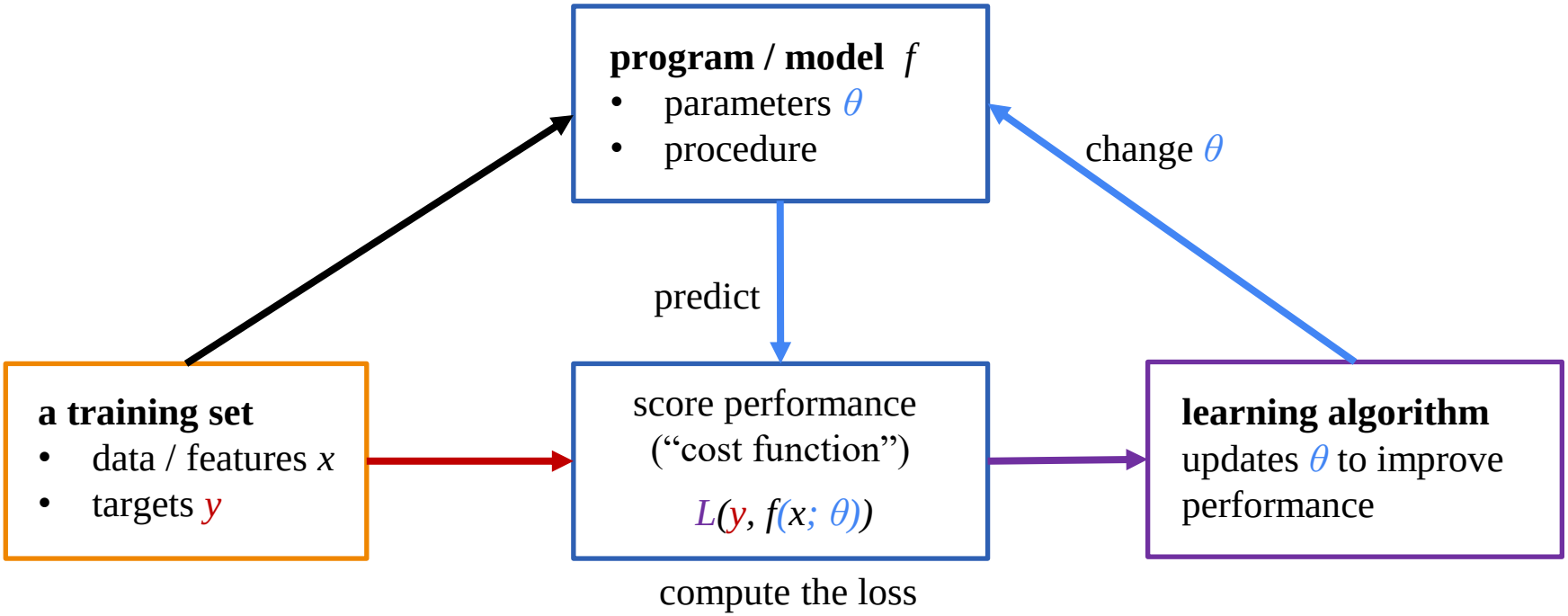
Keys: per-input vectors computed from inputs. Similarity to query determines per-input weights.

Values: per-input vectors from inputs used to compute output.



Review

- Linear Separability
- Multi-Layer Perceptron
- Backpropagation
- Convolutional Neural Networks and Transformers



Hands-on exercise

https://github.com/aimerykong/OIST-mini-course-CVML

Today's coding exercise:

- Understand the power of deep learning
- Understand the why deep learning produces powerful features
- Train a deep neural network for image classification

- Download the zip file (the same one as last week!) and upload to your gdrive
- Download and upload the jupyter notebook "code004_DNN_scratch.ipynb" to colab